somevariablestuff

April 14, 2026

1 LendingClub discrete-time hazard model project

2 File: data/raw/accepted_2007_to_2018Q4.csv.gz

3

4 Workflow:

5 1. Load and clean data

6 2. Define event and censoring

7 3. Create duration

8 4. Expand to loan-month panel

9 5. Train/valid/test split

10 6. Logistic hazard model

11 7. Random forest comparison

12 8. Cumulative PD

13 9. Expected loss / reserve / pricing summaries

[96]:
```
import warnings
warnings.filterwarnings("ignore")

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from pathlib import Path

from sklearn.compose import ColumnTransformer
```

```python
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import roc_auc_score, average_precision_score

pd.set_option("display.max_columns", 200)
pd.set_option("display.max_rows", 100)
```

[97]:
```python
DATA_DIR = Path("data")
RAW_DIR = DATA_DIR / "raw"
PROCESSED_DIR = DATA_DIR / "processed"
PROCESSED_DIR.mkdir(parents=True, exist_ok=True)

accepted_path = RAW_DIR / "lendingclub" /"accepted_2007_to_2018Q4.csv.gz"

# Runtime controls
USE_SAMPLE = True          # set to False if you want the full dataset
SAMPLE_SIZE = 50000        # increase later if your machine can handle it
RANDOM_STATE = 42
```

[98]:
```python
df_raw = pd.read_csv(
    accepted_path,
    low_memory=False,
    compression="gzip"
)

print("Raw shape:", df_raw.shape)
print("First 20 columns:", df_raw.columns.tolist()[:20])
```

```
Raw shape: (2260701, 151)
First 20 columns: ['id', 'member_id', 'loan_amnt', 'funded_amnt',
'funded_amnt_inv', 'term', 'int_rate', 'installment', 'grade', 'sub_grade',
'emp_title', 'emp_length', 'home_ownership', 'annual_inc',
'verification_status', 'issue_d', 'loan_status', 'pymnt_plan', 'url', 'desc']
```

[99]:
```python
key_columns = [
    "id",
    "issue_d",
    "loan_status",
    "term",
    "loan_amnt",
    "installment",
    "emp_length",
    "home_ownership",
    "annual_inc",
    "verification_status",
```

```
    "purpose",
    "addr_state",
    "dti",
    "delinq_2yrs",
    "inq_last_6mths",
    "open_acc",
    "pub_rec",
    "revol_bal",
    "revol_util",
    "total_acc",
    "mort_acc",
    "pub_rec_bankruptcies",
    "fico_range_low",
    "fico_range_high",
    "earliest_cr_line",
    "last_pymnt_d",
    "last_credit_pull_d",
]

missing_cols = [c for c in key_columns if c not in df_raw.columns]
print("Missing columns:", missing_cols)
```

```
Missing columns: []
```

[100]:
```
cols = [c for c in key_columns if c in df_raw.columns]
df = df_raw[cols].copy()

print("Working shape:", df.shape)
df.head()
```

```
Working shape: (2260701, 27)
```

[100]:
```
         id   issue_d loan_status        term  loan_amnt  installment  \
0  68407277  Dec-2015  Fully Paid   36 months     3600.0       123.03
1  68355089  Dec-2015  Fully Paid   36 months    24700.0       820.28
2  68341763  Dec-2015  Fully Paid   60 months    20000.0       432.66
3  66310712  Dec-2015     Current   60 months    35000.0       829.90
4  68476807  Dec-2015  Fully Paid   60 months    10400.0       289.91

  emp_length home_ownership  annual_inc verification_status  \
0  10+ years       MORTGAGE     55000.0        Not Verified
1  10+ years       MORTGAGE     65000.0        Not Verified
2  10+ years       MORTGAGE     63000.0        Not Verified
3  10+ years       MORTGAGE    110000.0     Source Verified
4    3 years       MORTGAGE    104433.0     Source Verified

              purpose addr_state    dti  delinq_2yrs  inq_last_6mths  \
0  debt_consolidation         PA   5.91          0.0             1.0
```

```
1      small_business         SD  16.06          1.0             4.0
2    home_improvement         IL  10.78          0.0             0.0
3  debt_consolidation         NJ  17.06          0.0             0.0
4      major_purchase         PA  25.37          1.0             3.0

   open_acc  pub_rec  revol_bal  revol_util  total_acc  mort_acc  \
0       7.0      0.0     2765.0        29.7       13.0       1.0
1      22.0      0.0    21470.0        19.2       38.0       4.0
2       6.0      0.0     7869.0        56.2       18.0       5.0
3      13.0      0.0     7802.0        11.6       17.0       1.0
4      12.0      0.0    21929.0        64.5       35.0       6.0

   pub_rec_bankruptcies  fico_range_low  fico_range_high earliest_cr_line  \
0                   0.0           675.0            679.0         Aug-2003
1                   0.0           715.0            719.0         Dec-1999
2                   0.0           695.0            699.0         Aug-2000
3                   0.0           785.0            789.0         Sep-2008
4                   0.0           695.0            699.0         Jun-1998

  last_pymnt_d last_credit_pull_d
0     Jan-2019           Mar-2019
1     Jun-2016           Mar-2019
2     Jun-2017           Mar-2019
3     Feb-2019           Mar-2019
4     Jul-2016           Mar-2018
```

[101]:
```
if USE_SAMPLE:
    df = df.sample(SAMPLE_SIZE, random_state=RANDOM_STATE).copy()

print("Shape after optional sampling:", df.shape)
```

```
Shape after optional sampling: (50000, 27)
```

[102]:
```
date_cols = ["issue_d", "earliest_cr_line", "last_pymnt_d",␣
 ↪"last_credit_pull_d"]

for c in date_cols:
    if c in df.columns:
        df[c] = pd.to_datetime(df[c], format="%b-%Y", errors="coerce")

df["fico_avg"] = (df["fico_range_low"] + df["fico_range_high"]) / 2

df["term_num"] = (
    df["term"]
    .astype(str)
    .str.extract(r"(\d+)")
    .astype(float)
```

```
)

df["credit_history_years"] = (
    (df["issue_d"] - df["earliest_cr_line"]).dt.days / 365.25
)

df[["issue_d", "loan_status", "term", "term_num", "fico_avg",␣
 ↪"credit_history_years"]].head()
```

[102]:

```
           issue_d loan_status        term  term_num  fico_avg  \
392949  2015-02-01     Current   60 months      60.0     737.0
1273506 2014-05-01  Fully Paid   36 months      36.0     687.0
324024  2015-04-01  Fully Paid   36 months      36.0     782.0
2066630 2017-12-01  Fully Paid   36 months      36.0     742.0
477199  2018-02-01     Current   60 months      60.0     717.0

         credit_history_years
392949              33.336071
1273506             13.081451
324024              11.581109
2066630             12.501027
477199              18.590007
```

[103]:

```
status_counts = df["loan_status"].value_counts(dropna=False)
status_counts
```

[103]:

```
loan_status
Fully Paid                                          23771
Current                                             19370
Charged Off                                          6015
Late (31-120 days)                                    505
In Grace Period                                       186
Late (16-30 days)                                     108
Does not meet the credit policy. Status:Fully Paid     24
Does not meet the credit policy. Status:Charged Off    19
NaN                                                     2
Name: count, dtype: int64
```

[104]:

```
# Basic event definition:
# event = 1 for default-like outcomes
# event = 0 otherwise

default_statuses = {
    "Charged Off",
    "Default",
}
```

```
df["event"] = df["loan_status"].isin(default_statuses).astype(int)

# Use last payment date if available; otherwise use last credit pull date
df["end_date"] = df["last_pymnt_d"]
mask_missing_end = df["end_date"].isna()
df.loc[mask_missing_end, "end_date"] = df.loc[mask_missing_end,␣
↪"last_credit_pull_d"]

df[["loan_status", "event", "issue_d", "last_pymnt_d", "last_credit_pull_d",␣
↪"end_date"]].head()
```

[104]:

```
        loan_status  event    issue_d last_pymnt_d last_credit_pull_d  \
392949      Current      0 2015-02-01   2019-03-01         2019-03-01
1273506  Fully Paid      0 2014-05-01   2017-06-01         2017-08-01
324024   Fully Paid      0 2015-04-01   2017-03-01         2019-02-01
2066630  Fully Paid      0 2017-12-01   2018-10-01         2018-10-01
477199      Current      0 2018-02-01   2019-03-01         2019-03-01

          end_date
392949  2019-03-01
1273506 2017-06-01
324024  2017-03-01
2066630 2018-10-01
477199  2019-03-01
```

[105]:

```
def month_diff(start, end):
    return (end.dt.year - start.dt.year) * 12 + (end.dt.month - start.dt.month)

df = df[df["issue_d"].notna()].copy()
df = df[df["end_date"].notna()].copy()

df["duration_months"] = month_diff(df["issue_d"], df["end_date"])
df["duration_months"] = df["duration_months"].clip(lower=1)

print(df.shape)
df[["id", "issue_d", "end_date", "duration_months", "event"]].head()
```

```
(49998, 33)
```

[105]:

```
                id    issue_d   end_date  duration_months  event
392949    39651438 2015-02-01 2019-03-01               49      0
1273506   16411620 2014-05-01 2017-06-01               37      0
324024    45122316 2015-04-01 2017-03-01               23      0
2066630  125356772 2017-12-01 2018-10-01               10      0
477199   128490686 2018-02-01 2019-03-01               13      0
```

[106]:
```
df = df[df["term_num"].isin([36, 60])].copy()
df = df[df["duration_months"] >= 1].copy()
df = df[df["fico_avg"].notna()].copy()

print("After cleaning/filtering:", df.shape)
```

```
After cleaning/filtering: (49998, 33)
```

[107]:
```
print("Event rate:")
print(df["event"].value_counts(normalize=True))

print("\nLoan status counts:")
print(df["loan_status"].value_counts(dropna=False).head(20))

print("\nDuration summary:")
print(df["duration_months"].describe())
```

```
Event rate:
event
0    0.879695
1    0.120305
Name: proportion, dtype: float64

Loan status counts:
loan_status
Fully Paid                                            23771
Current                                               19370
Charged Off                                            6015
Late (31-120 days)                                      505
In Grace Period                                         186
Late (16-30 days)                                       108
Does not meet the credit policy. Status:Fully Paid       24
Does not meet the credit policy. Status:Charged Off      19
Name: count, dtype: int64

Duration summary:
count    49998.000000
mean        19.577083
std         12.398375
min          1.000000
25%          9.000000
50%         17.000000
75%         29.000000
max         70.000000
Name: duration_months, dtype: float64
```

[108]:
```
loan_level_path = PROCESSED_DIR / "loan_level_survival_base.csv"
df.to_csv(loan_level_path, index=False)
```

```
print(f"Saved loan-level base to: {loan_level_path}")
```

```
Saved loan-level base to: data/processed/loan_level_survival_base.csv
```

[109]:
```
static_cols = [
    "loan_amnt",
    "term_num",
    "installment",
    "emp_length",
    "home_ownership",
    "annual_inc",
    "verification_status",
    "purpose",
    "addr_state",
    "dti",
    "delinq_2yrs",
    "inq_last_6mths",
    "open_acc",
    "pub_rec",
    "revol_bal",
    "revol_util",
    "total_acc",
    "mort_acc",
    "pub_rec_bankruptcies",
    "fico_avg",
    "credit_history_years",
]

def expand_loan(row):
    n = int(row["duration_months"])
    event = int(row["event"])

    out = pd.DataFrame({
        "id": row["id"],
        "issue_d": row["issue_d"],
        "month_on_book": np.arange(1, n + 1),
        "duration_months": row["duration_months"],
        "event": row["event"],
    })

    out["hazard_target"] = 0
    if event == 1:
        out.loc[out["month_on_book"] == n, "hazard_target"] = 1

    for col in static_cols:
        out[col] = row[col]
```

```
    return out
```

[110]:
```
panel_parts = []
for _, row in df.iterrows():
    panel_parts.append(expand_loan(row))

panel = pd.concat(panel_parts, ignore_index=True)

print("Panel shape:", panel.shape)
panel.head()
```

```
Panel shape: (978815, 27)
```

[110]:
```
         id    issue_d  month_on_book  duration_months  event  hazard_target  \
0  39651438 2015-02-01              1               49      0              0
1  39651438 2015-02-01              2               49      0              0
2  39651438 2015-02-01              3               49      0              0
3  39651438 2015-02-01              4               49      0              0
4  39651438 2015-02-01              5               49      0              0

   loan_amnt  term_num  installment emp_length home_ownership  annual_inc  \
0    32000.0      60.0       687.65  10+ years       MORTGAGE    120000.0
1    32000.0      60.0       687.65  10+ years       MORTGAGE    120000.0
2    32000.0      60.0       687.65  10+ years       MORTGAGE    120000.0
3    32000.0      60.0       687.65  10+ years       MORTGAGE    120000.0
4    32000.0      60.0       687.65  10+ years       MORTGAGE    120000.0

  verification_status             purpose addr_state    dti  delinq_2yrs  \
0            Verified  debt_consolidation         CA  24.05          0.0
1            Verified  debt_consolidation         CA  24.05          0.0
2            Verified  debt_consolidation         CA  24.05          0.0
3            Verified  debt_consolidation         CA  24.05          0.0
4            Verified  debt_consolidation         CA  24.05          0.0

   inq_last_6mths  open_acc  pub_rec  revol_bal  revol_util  total_acc  \
0             0.0      20.0      0.0    39687.0        57.8       42.0
1             0.0      20.0      0.0    39687.0        57.8       42.0
2             0.0      20.0      0.0    39687.0        57.8       42.0
3             0.0      20.0      0.0    39687.0        57.8       42.0
4             0.0      20.0      0.0    39687.0        57.8       42.0

   mort_acc  pub_rec_bankruptcies  fico_avg  credit_history_years
0       2.0                   0.0     737.0             33.336071
1       2.0                   0.0     737.0             33.336071
2       2.0                   0.0     737.0             33.336071
3       2.0                   0.0     737.0             33.336071
4       2.0                   0.0     737.0             33.336071
```

[111]:
```
panel["mob_6bin"] = pd.cut(
    panel["month_on_book"],
    bins=[0, 6, 12, 24, 36, 60, 120],
    labels=["1_6", "7_12", "13_24", "25_36", "37_60", "61_plus"]
)

panel["month_on_book_sq"] = panel["month_on_book"] ** 2

panel[["id", "month_on_book", "mob_6bin", "hazard_target"]].head(20)
```

[111]:

	id	month_on_book	mob_6bin	hazard_target
0	39651438	1	1_6	0
1	39651438	2	1_6	0
2	39651438	3	1_6	0
3	39651438	4	1_6	0
4	39651438	5	1_6	0
5	39651438	6	1_6	0
6	39651438	7	7_12	0
7	39651438	8	7_12	0
8	39651438	9	7_12	0
9	39651438	10	7_12	0
10	39651438	11	7_12	0
11	39651438	12	7_12	0
12	39651438	13	13_24	0
13	39651438	14	13_24	0
14	39651438	15	13_24	0
15	39651438	16	13_24	0
16	39651438	17	13_24	0
17	39651438	18	13_24	0
18	39651438	19	13_24	0
19	39651438	20	13_24	0

[112]:
```
# More flexible time features + interactions

panel["mob_str"] = panel["month_on_book"].astype(str)

panel["fico_x_mob"] = panel["fico_avg"] * panel["month_on_book"]
panel["term_x_mob"] = panel["term_num"] * panel["month_on_book"]
panel["dti_x_mob"] = panel["dti"] * panel["month_on_book"]

panel[[
    "id", "month_on_book", "mob_str",
    "fico_avg", "fico_x_mob",
    "term_num", "term_x_mob",
    "dti", "dti_x_mob"
]].head()
```

[112]:

```
          id  month_on_book mob_str  fico_avg  fico_x_mob  term_num  \
0  39651438              1       1     737.0       737.0      60.0
1  39651438              2       2     737.0      1474.0      60.0
2  39651438              3       3     737.0      2211.0      60.0
3  39651438              4       4     737.0      2948.0      60.0
4  39651438              5       5     737.0      3685.0      60.0

   term_x_mob    dti  dti_x_mob
0        60.0  24.05      24.05
1       120.0  24.05      48.10
2       180.0  24.05      72.15
3       240.0  24.05      96.20
4       300.0  24.05     120.25
```

[113]:

```python
panel_path = PROCESSED_DIR / "loan_month_panel.csv"
panel.to_csv(panel_path, index=False)
print(f"Saved panel to: {panel_path}")
```

```
Saved panel to: data/processed/loan_month_panel.csv
```

[114]:

```python
train_panel = panel[panel["issue_d"] < "2016-01-01"].copy()
valid_panel = panel[(panel["issue_d"] >= "2016-01-01") & (panel["issue_d"] <␣
↪"2017-01-01")].copy()
test_panel  = panel[panel["issue_d"] >= "2017-01-01"].copy()

print("Train shape:", train_panel.shape)
print("Valid shape:", valid_panel.shape)
print("Test shape :", test_panel.shape)
```

```
Train shape: (517939, 33)
Valid shape: (218609, 33)
Test shape : (242267, 33)
```

[115]:

```python
target = "hazard_target"

numeric_features = [
    "loan_amnt",
    "term_num",
    "installment",
    "annual_inc",
    "dti",
    "delinq_2yrs",
    "inq_last_6mths",
    "open_acc",
    "pub_rec",
    "revol_bal",
    "revol_util",
    "total_acc",
```

```
    "mort_acc",
    "pub_rec_bankruptcies",
    "fico_avg",
    "credit_history_years",
    "fico_x_mob",
    "term_x_mob",
    "dti_x_mob",
]

categorical_features = [
    "emp_length",
    "home_ownership",
    "verification_status",
    "purpose",
    "addr_state",
    "mob_str",       # full month dummy
    "mob_6bin",      # can keep this too
]
```

[116]:
```
X_train = train_panel[numeric_features + categorical_features]
y_train = train_panel[target]

X_valid = valid_panel[numeric_features + categorical_features]
y_valid = valid_panel[target]

X_test = test_panel[numeric_features + categorical_features]
y_test = test_panel[target]

print(X_train.shape, y_train.shape)
print(X_valid.shape, y_valid.shape)
print(X_test.shape, y_test.shape)
```

```
(517939, 26) (517939,)
(218609, 26) (218609,)
(242267, 26) (242267,)
```

[117]:
```
numeric_transformer = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler()),
])

categorical_transformer = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore")),
])

preprocessor = ColumnTransformer([
    ("num", numeric_transformer, numeric_features),
```

```
    ("cat", categorical_transformer, categorical_features),
])
```

[118]:
```
hazard_model = Pipeline([
    ("prep", preprocessor),
    ("model", LogisticRegression(max_iter=2000))
])

hazard_model.fit(X_train, y_train)
```

[118]:
```
Pipeline(steps=[('prep',
                 ColumnTransformer(transformers=[('num',
                                                  Pipeline(steps=[('imputer',
SimpleImputer(strategy='median')),
                                                                  ('scaler',
StandardScaler())]),
                                                  ['loan_amnt', 'term_num',
                                                   'installment', 'annual_inc',
                                                   'dti', 'delinq_2yrs',
                                                   'inq_last_6mths', 'open_acc',
                                                   'pub_rec', 'revol_bal',
                                                   'revol_util', 'total_acc',
                                                   'mort_acc',
                                                   'pub_rec_bankruptcies',
                                                   'fico_avg',
                                                   'credit_history_years',
                                                   'fico_x_mob', 'term_x_mob',
                                                   'dti_x_mob']),
                                                 ('cat',
                                                  Pipeline(steps=[('imputer',
SimpleImputer(strategy='most_frequent')),
                                                                  ('onehot',
OneHotEncoder(handle_unknown='ignore'))]),
                                                  ['emp_length',
                                                   'home_ownership',
                                                   'verification_status',
                                                   'purpose', 'addr_state',
                                                   'mob_str', 'mob_6bin'])])),
                 ('model', LogisticRegression(max_iter=2000))])
```

[119]:
```
valid_pred_hazard = hazard_model.predict_proba(X_valid)[:, 1]
test_pred_hazard = hazard_model.predict_proba(X_test)[:, 1]

hazard_results = {
    "valid_roc_auc": roc_auc_score(y_valid, valid_pred_hazard),
    "valid_pr_auc": average_precision_score(y_valid, valid_pred_hazard),
    "test_roc_auc": roc_auc_score(y_test, test_pred_hazard),
```

```
    "test_pr_auc": average_precision_score(y_test, test_pred_hazard),
}

hazard_results
```

[119]:
```
{'valid_roc_auc': 0.6707368973775432,
 'valid_pr_auc': 0.014084952061406895,
 'test_roc_auc': 0.6764559575734034,
 'test_pr_auc': 0.008912978406677879}
```

[120]:
```
from sklearn.calibration import CalibratedClassifierCV
```

[121]:
```
rf_base = Pipeline([
    ("prep", preprocessor),
    ("model", RandomForestClassifier(
        n_estimators=200,
        min_samples_leaf=50,
        random_state=RANDOM_STATE,
        n_jobs=-1,
        class_weight="balanced_subsample"
    )),
])

rf_calibrated = CalibratedClassifierCV(
    estimator=rf_base,
    method="sigmoid",
    cv=3
)

rf_calibrated.fit(X_train, y_train)
```

[121]:
```
CalibratedClassifierCV(cv=3,
                       estimator=Pipeline(steps=[('prep',
ColumnTransformer(transformers=[('num',
   Pipeline(steps=[('imputer',
                    SimpleImputer(strategy='median')),
                   ('scaler',
                    StandardScaler())]),
   ['loan_amnt',
    'term_num',
    'installment',
    'annual_inc',
    'dti',
    'delinq_2yrs',
    'inq_last_6mths',
    'open_acc',
    'pub_rec',
```

```
     'revol_bal',
     'revol_util',
     'total_acc',
     'mort_acc',
     '…
    Pipeline(steps=[('imputer',
                     SimpleImputer(strategy='most_frequent')),
                    ('onehot',
                     OneHotEncoder(handle_unknown='ignore'))]),
    ['emp_length',
     'home_ownership',
     'verification_status',
     'purpose',
     'addr_state',
     'mob_str',
     'mob_6bin'])])),
                                                    ('model',
RandomForestClassifier(class_weight='balanced_subsample',
min_samples_leaf=50,
n_estimators=200,
n_jobs=-1,
random_state=42))]))
```

[122]:
```
valid_pred_rf = rf_calibrated.predict_proba(X_valid)[:, 1]
test_pred_rf = rf_calibrated.predict_proba(X_test)[:, 1]

rf_results = {
    "valid_roc_auc": roc_auc_score(y_valid, valid_pred_rf),
    "valid_pr_auc": average_precision_score(y_valid, valid_pred_rf),
    "test_roc_auc": roc_auc_score(y_test, test_pred_rf),
    "test_pr_auc": average_precision_score(y_test, test_pred_rf),
}

rf_results
```

[122]:
```
{'valid_roc_auc': 0.6509492518507828,
 'valid_pr_auc': 0.01297307713652124,
 'test_roc_auc': 0.6668219845528852,
 'test_pr_auc': 0.008616006069964852}
```

[123]:
```
results_df = pd.DataFrame([
    {
        "model": "logistic_hazard",
        **hazard_results
    },
    {
        "model": "random_forest",
```

```
        **rf_results
    }
])

results_df
```

[123]:

	model	valid_roc_auc	valid_pr_auc	test_roc_auc	test_pr_auc
0	logistic_hazard	0.670737	0.014085	0.676456	0.008913
1	random_forest	0.650949	0.012973	0.666822	0.008616

[124]:

```
test_scored = test_panel.copy()
test_scored["pred_hazard_logit"] = test_pred_hazard
test_scored["pred_hazard_rf"] = test_pred_rf

test_scored = test_scored.sort_values(["id", "month_on_book"]).copy()
test_scored.head()
```

[124]:

```
               id    issue_d  month_on_book  duration_months  event  \
214997  100001365 2017-03-01              1               24      0
214998  100001365 2017-03-01              2               24      0
214999  100001365 2017-03-01              3               24      0
215000  100001365 2017-03-01              4               24      0
215001  100001365 2017-03-01              5               24      0

        hazard_target  loan_amnt  term_num  installment emp_length  \
214997              0    20000.0      36.0       683.46  10+ years
214998              0    20000.0      36.0       683.46  10+ years
214999              0    20000.0      36.0       683.46  10+ years
215000              0    20000.0      36.0       683.46  10+ years
215001              0    20000.0      36.0       683.46  10+ years

       home_ownership  annual_inc verification_status             purpose  \
214997       MORTGAGE     87000.0        Not Verified  debt_consolidation
214998       MORTGAGE     87000.0        Not Verified  debt_consolidation
214999       MORTGAGE     87000.0        Not Verified  debt_consolidation
215000       MORTGAGE     87000.0        Not Verified  debt_consolidation
215001       MORTGAGE     87000.0        Not Verified  debt_consolidation

       addr_state   dti  delinq_2yrs  inq_last_6mths  open_acc  pub_rec  \
214997         MT  25.3          0.0             0.0      12.0      0.0
214998         MT  25.3          0.0             0.0      12.0      0.0
214999         MT  25.3          0.0             0.0      12.0      0.0
215000         MT  25.3          0.0             0.0      12.0      0.0
215001         MT  25.3          0.0             0.0      12.0      0.0

        revol_bal  revol_util  total_acc  mort_acc  pub_rec_bankruptcies  \
214997    19262.0        79.3       19.0       1.0                   0.0
```

```
214998    19262.0        79.3       19.0       1.0                   0.0
214999    19262.0        79.3       19.0       1.0                   0.0
215000    19262.0        79.3       19.0       1.0                   0.0
215001    19262.0        79.3       19.0       1.0                   0.0

        fico_avg  credit_history_years mob_6bin  month_on_book_sq mob_str  \
214997     662.0             12.495551      1_6                 1       1
214998     662.0             12.495551      1_6                 4       2
214999     662.0             12.495551      1_6                 9       3
215000     662.0             12.495551      1_6                16       4
215001     662.0             12.495551      1_6                25       5

        fico_x_mob  term_x_mob  dti_x_mob  pred_hazard_logit  pred_hazard_rf
214997       662.0        36.0       25.3           0.002230        0.002457
214998      1324.0        72.0       50.6           0.002911        0.002960
214999      1986.0       108.0       75.9           0.003722        0.003993
215000      2648.0       144.0      101.2           0.003975        0.003728
215001      3310.0       180.0      126.5           0.004296        0.003823
```

[125]:
```python
test_scored["survival_prob_logit"] = test_scored.
 ↪groupby("id")["pred_hazard_logit"].transform(
    lambda x: (1 - x).cumprod()
)
test_scored["cum_pd_logit"] = 1 - test_scored["survival_prob_logit"]

test_scored["survival_prob_rf"] = test_scored.groupby("id")["pred_hazard_rf"].
 ↪transform(
    lambda x: (1 - x).cumprod()
)
test_scored["cum_pd_rf"] = 1 - test_scored["survival_prob_rf"]

test_scored[[
    "id", "month_on_book", "pred_hazard_logit", "cum_pd_logit",
    "pred_hazard_rf", "cum_pd_rf"
]].head(20)
```

[125]:
```
               id  month_on_book  pred_hazard_logit  cum_pd_logit  \
214997  100001365              1           0.002230      0.002230
214998  100001365              2           0.002911      0.005135
214999  100001365              3           0.003722      0.008838
215000  100001365              4           0.003975      0.012778
215001  100001365              5           0.004296      0.017019
215002  100001365              6           0.004810      0.021747
215003  100001365              7           0.007656      0.029237
215004  100001365              8           0.006805      0.035843
215005  100001365              9           0.007023      0.042614
215006  100001365             10           0.007455      0.049751
```

```
215007  100001365            11          0.008066      0.057415
215008  100001365            12          0.007959      0.064918
215009  100001365            13          0.009168      0.073490
215010  100001365            14          0.007742      0.080663
215011  100001365            15          0.007892      0.087918
215012  100001365            16          0.008689      0.095843
215013  100001365            17          0.009937      0.104828
215014  100001365            18          0.008962      0.112850
215015  100001365            19          0.008128      0.120061
215016  100001365            20          0.009887      0.128761

        pred_hazard_rf  cum_pd_rf
214997        0.002457   0.002457
214998        0.002960   0.005410
214999        0.003993   0.009381
215000        0.003728   0.013074
215001        0.003823   0.016847
215002        0.004532   0.021303
215003        0.006303   0.027472
215004        0.006694   0.033983
215005        0.007199   0.040937
215006        0.007616   0.048241
215007        0.008972   0.056780
215008        0.008367   0.064672
215009        0.008355   0.072487
215010        0.009459   0.081261
215011        0.009156   0.089673
215012        0.011827   0.100439
215013        0.012349   0.111548
215014        0.010365   0.120757
215015        0.010761   0.130218
215016        0.010912   0.139709
```

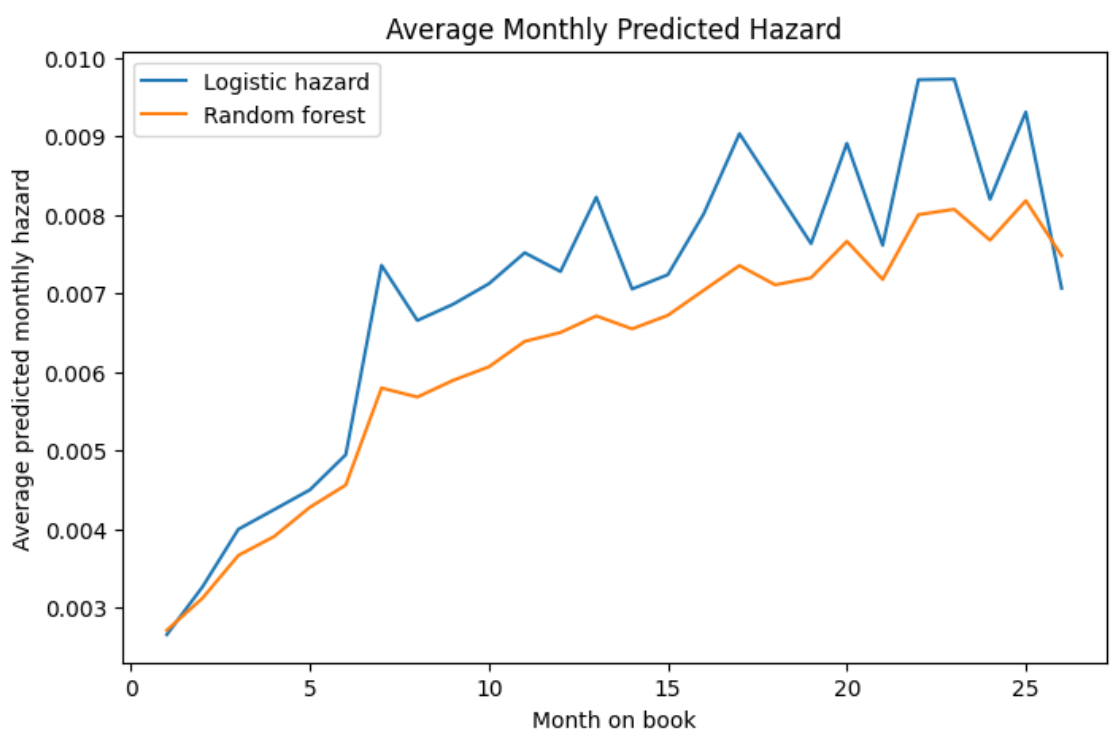
[126]:
```
pd_summary = (
    test_scored[test_scored["month_on_book"].isin([6, 12, 24, 36])]
    .groupby("month_on_book")
    .agg(
        avg_cum_pd_logit=("cum_pd_logit", "mean"),
        avg_cum_pd_rf=("cum_pd_rf", "mean"),
        loans=("id", "nunique"),
    )
    .reset_index()
)

pd_summary
```

[126]:

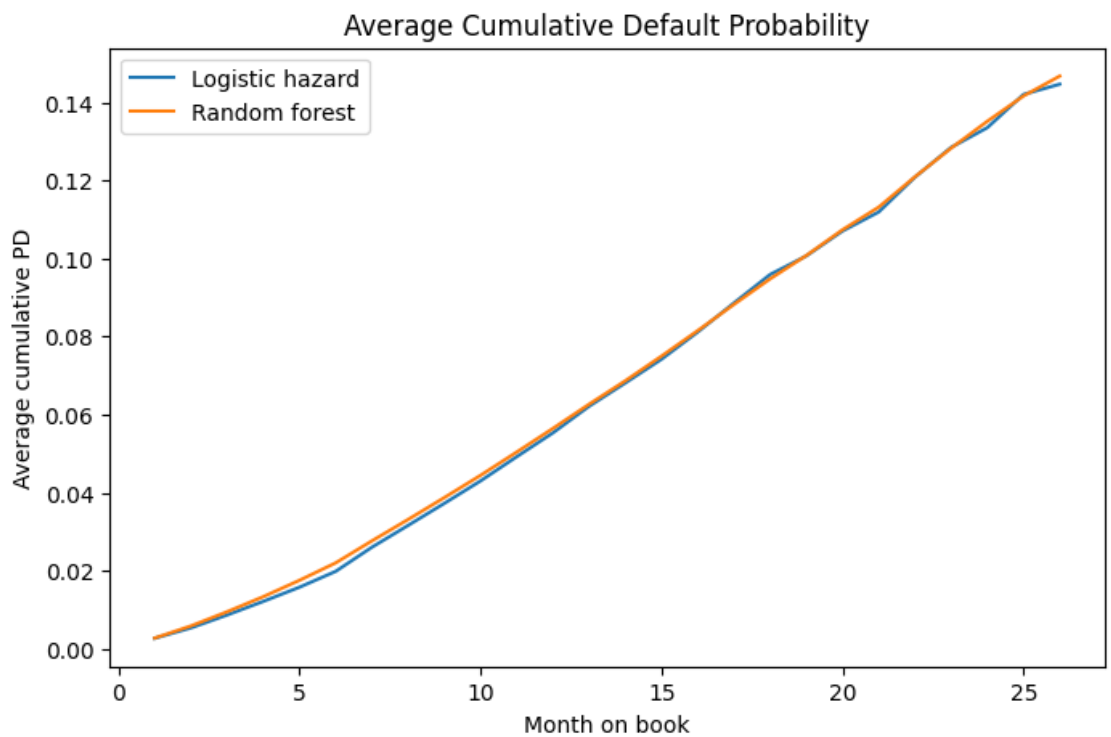
	month_on_book	avg_cum_pd_logit	avg_cum_pd_rf	loans
0	6	0.019812	0.021988	16402
1	12	0.055310	0.056448	9504
2	24	0.133549	0.135185	1103

[127]:

```
plot_hazard = (
    test_scored.groupby("month_on_book")[["pred_hazard_logit",␣
 ↪"pred_hazard_rf"]]
    .mean()
    .reset_index()
)

plt.figure(figsize=(8, 5))
plt.plot(plot_hazard["month_on_book"], plot_hazard["pred_hazard_logit"],␣
 ↪label="Logistic hazard")
plt.plot(plot_hazard["month_on_book"], plot_hazard["pred_hazard_rf"],␣
 ↪label="Random forest")
plt.xlabel("Month on book")
plt.ylabel("Average predicted monthly hazard")
plt.title("Average Monthly Predicted Hazard")
plt.legend()
plt.show()
```



[128]:
```
plot_cum_pd = (
    test_scored.groupby("month_on_book")[["cum_pd_logit", "cum_pd_rf"]]
    .mean()
    .reset_index()
)

plt.figure(figsize=(8, 5))
plt.plot(plot_cum_pd["month_on_book"], plot_cum_pd["cum_pd_logit"],␣
 ↪label="Logistic hazard")
plt.plot(plot_cum_pd["month_on_book"], plot_cum_pd["cum_pd_rf"], label="Random␣
 ↪forest")
plt.xlabel("Month on book")
plt.ylabel("Average cumulative PD")
plt.title("Average Cumulative Default Probability")
plt.legend()
plt.show()
```



[129]:
```
loan_12m = test_scored[test_scored["month_on_book"] == 12].copy()

print("Loans with 12-month horizon rows:", loan_12m["id"].nunique())
loan_12m[["id", "cum_pd_logit", "cum_pd_rf"]].head()
```

```
Loans with 12-month horizon rows: 9504
```

[129]:

```
                id  cum_pd_logit  cum_pd_rf
215008  100001365      0.064918   0.064672
539498  100001532      0.019600   0.038749
62778   100002096      0.066961   0.057048
86881   100003912      0.030972   0.059209
866480  100005538      0.056918   0.067245
```

[130]:

```
LGD = 0.60

loan_12m["ead_proxy"] = loan_12m["loan_amnt"]
loan_12m["expected_loss_12m_logit"] = loan_12m["cum_pd_logit"] * LGD *␣
↪loan_12m["ead_proxy"]
loan_12m["expected_loss_12m_rf"] = loan_12m["cum_pd_rf"] * LGD *␣
↪loan_12m["ead_proxy"]

loan_12m[[
    "id",
    "loan_amnt",
    "cum_pd_logit",
    "cum_pd_rf",
    "expected_loss_12m_logit",
    "expected_loss_12m_rf"
]].head()
```

[130]:

```
                id  loan_amnt  cum_pd_logit  cum_pd_rf  \
215008  100001365    20000.0      0.064918   0.064672
539498  100001532    30000.0      0.019600   0.038749
62778   100002096    20000.0      0.066961   0.057048
86881   100003912    28500.0      0.030972   0.059209
866480  100005538    14400.0      0.056918   0.067245

        expected_loss_12m_logit  expected_loss_12m_rf
215008               779.011180            776.069824
539498               352.802225            697.481695
62778                803.531124            684.571694
86881                529.615718           1012.482208
866480               491.772543            580.993074
```

[131]:

```
funding_cost = 0.04
profit_margin = 0.03

loan_12m["el_rate_12m_logit"] = loan_12m["expected_loss_12m_logit"] /␣
↪loan_12m["loan_amnt"]
loan_12m["el_rate_12m_rf"] = loan_12m["expected_loss_12m_rf"] /␣
↪loan_12m["loan_amnt"]
```

```
loan_12m["required_rate_logit"] = funding_cost + profit_margin +␣
↪loan_12m["el_rate_12m_logit"]
loan_12m["required_rate_rf"] = funding_cost + profit_margin +␣
↪loan_12m["el_rate_12m_rf"]

loan_12m[[
    "id",
    "required_rate_logit",
    "required_rate_rf"
]].head()
```

[131]:

	id	required_rate_logit	required_rate_rf
215008	100001365	0.108951	0.108803
539498	100001532	0.081760	0.093249
62778	100002096	0.110177	0.104229
86881	100003912	0.088583	0.105526
866480	100005538	0.104151	0.110347

[132]:

```
loan_12m["risk_band_logit"] = pd.qcut(
    loan_12m["cum_pd_logit"],
    q=5,
    labels=["Very Low", "Low", "Medium", "High", "Very High"]
)

reserve_summary_logit = (
    loan_12m.groupby("risk_band_logit")
    .agg(
        loans=("id", "count"),
        avg_pd_12m=("cum_pd_logit", "mean"),
        total_expected_loss=("expected_loss_12m_logit", "sum"),
        avg_required_rate=("required_rate_logit", "mean"),
    )
    .reset_index()
)

reserve_summary_logit
```

[132]:

	risk_band_logit	loans	avg_pd_12m	total_expected_loss	avg_required_rate
0	Very Low	1901	0.019531	3.259416e+05	0.081719
1	Low	1901	0.034015	5.386392e+05	0.090409
2	Medium	1900	0.046647	7.476895e+05	0.097988
3	High	1901	0.063327	1.076963e+06	0.107996
4	Very High	1901	0.113025	2.367102e+06	0.137815

[133]:

```
loan_12m["risk_band_rf"] = pd.qcut(
    loan_12m["cum_pd_rf"],
    q=5,
```

```
    labels=["Very Low", "Low", "Medium", "High", "Very High"]
)

reserve_summary_rf = (
    loan_12m.groupby("risk_band_rf")
    .agg(
        loans=("id", "count"),
        avg_pd_12m=("cum_pd_rf", "mean"),
        total_expected_loss=("expected_loss_12m_rf", "sum"),
        avg_required_rate=("required_rate_rf", "mean"),
    )
    .reset_index()
)

reserve_summary_rf
```

[133]:

	risk_band_rf	loans	avg_pd_12m	total_expected_loss	avg_required_rate
0	Very Low	1901	0.036443	5.814805e+05	0.091866
1	Low	1901	0.045021	7.298987e+05	0.097013
2	Medium	1900	0.052736	8.723510e+05	0.101642
3	High	1901	0.062676	1.093686e+06	0.107605
4	Very High	1901	0.085365	1.711478e+06	0.121219

[134]:

```
test_scored_path = PROCESSED_DIR / "test_panel_scored.csv"
loan_12m_path = PROCESSED_DIR / "loan_level_12m_outputs.csv"
results_path = PROCESSED_DIR / "model_comparison_results.csv"

test_scored.to_csv(test_scored_path, index=False)
loan_12m.to_csv(loan_12m_path, index=False)
results_df.to_csv(results_path, index=False)

print(f"Saved scored panel to: {test_scored_path}")
print(f"Saved 12m loan outputs to: {loan_12m_path}")
print(f"Saved model comparison to: {results_path}")
```

```
Saved scored panel to: data/processed/test_panel_scored.csv
Saved 12m loan outputs to: data/processed/loan_level_12m_outputs.csv
Saved model comparison to: data/processed/model_comparison_results.csv
```

[135]:

```
print("Model comparison:")
display(results_df)

print("\n12-month cumulative PD summary:")
display(pd_summary)

print("\nReserve summary (logistic hazard):")
display(reserve_summary_logit)
```

```
Model comparison:

             model  valid_roc_auc  valid_pr_auc  test_roc_auc  test_pr_auc
0  logistic_hazard       0.670737      0.014085      0.676456     0.008913
1    random_forest       0.650949      0.012973      0.666822     0.008616


12-month cumulative PD summary:

   month_on_book  avg_cum_pd_logit  avg_cum_pd_rf  loans
0              6          0.019812       0.021988  16402
1             12          0.055310       0.056448   9504
2             24          0.133549       0.135185   1103


Reserve summary (logistic hazard):

  risk_band_logit  loans  avg_pd_12m  total_expected_loss  avg_required_rate
0        Very Low   1901    0.019531         3.259416e+05           0.081719
1             Low   1901    0.034015         5.386392e+05           0.090409
2          Medium   1900    0.046647         7.476895e+05           0.097988
3            High   1901    0.063327         1.076963e+06           0.107996
4       Very High   1901    0.113025         2.367102e+06           0.137815
```

[136]:
```python
loan_12m["risk_band_rf"] = pd.qcut(
    loan_12m["cum_pd_rf"],
    q=5,
    labels=["Very Low", "Low", "Medium", "High", "Very High"]
)

reserve_summary_rf = (
    loan_12m.groupby("risk_band_rf")
    .agg(
        loans=("id", "count"),
        avg_pd_12m=("cum_pd_rf", "mean"),
        total_expected_loss=("expected_loss_12m_rf", "sum"),
        avg_required_rate=("required_rate_rf", "mean"),
    )
    .reset_index()
)

reserve_summary_rf
```

[136]:
```
  risk_band_rf  loans  avg_pd_12m  total_expected_loss  avg_required_rate
0     Very Low   1901    0.036443         5.814805e+05           0.091866
1          Low   1901    0.045021         7.298987e+05           0.097013
2       Medium   1900    0.052736         8.723510e+05           0.101642
3         High   1901    0.062676         1.093686e+06           0.107605
4    Very High   1901    0.085365         1.711478e+06           0.121219
```

14 Looking at test

[137]:
```
test_loans = df[df["issue_d"] >= "2017-01-01"].copy()

loan_obs = test_loans[["id", "duration_months", "event"]].drop_duplicates().
 ↪copy()

print(loan_obs.shape)
loan_obs.head()
```

```
(20781, 3)
```

[137]:

	id	duration_months	event
2066630	125356772	10	0
477199	128490686	13	0
641990	109655666	14	0
507598	127049010	14	0
2129008	121746633	17	0

[138]:
```
max_mob = int(test_scored["month_on_book"].max())
months = np.arange(1, max_mob + 1)

n_loans = loan_obs["id"].nunique()

observed_curve = pd.DataFrame({"month_on_book": months})

observed_curve["observed_cum_pd"] = [
    ((loan_obs["event"] == 1) & (loan_obs["duration_months"] <= m)).sum() /␣
 ↪n_loans
    for m in months
]

observed_curve.head(15)
```

[138]:

	month_on_book	observed_cum_pd
0	1	0.001732
1	2	0.004283
2	3	0.007266
3	4	0.011693
4	5	0.015495
5	6	0.019200
6	7	0.023242
7	8	0.026659
8	9	0.030124
9	10	0.033396
10	11	0.036524
11	12	0.040133
12	13	0.042250

```
13              14          0.044608
14              15          0.046340
```

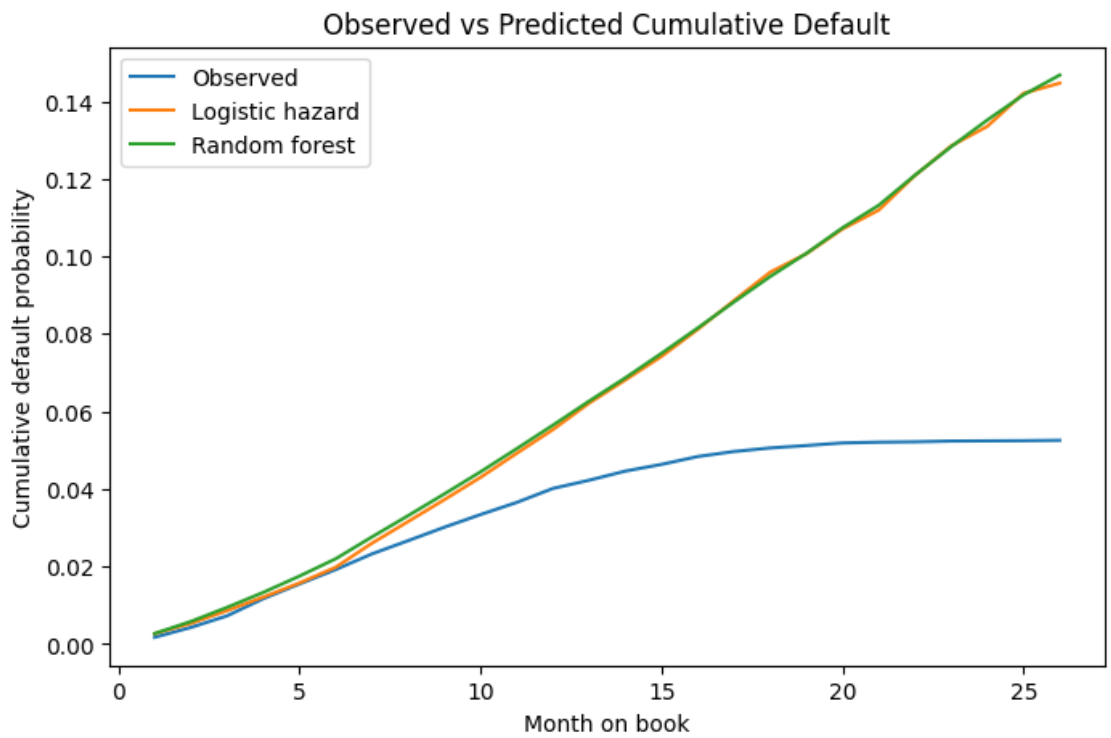
[139]:
```
plot_cum_pd = (
    test_scored.groupby("month_on_book")[["cum_pd_logit", "cum_pd_rf"]]
    .mean()
    .reset_index()
)
```

[140]:
```
compare_curve = observed_curve.merge(
    plot_cum_pd,
    on="month_on_book",
    how="inner"
)

compare_curve.head()
```

[140]:

	month_on_book	observed_cum_pd	cum_pd_logit	cum_pd_rf
0	1	0.001732	0.002654	0.002710
1	2	0.004283	0.005313	0.005819
2	3	0.007266	0.008619	0.009456
3	4	0.011693	0.012111	0.013319
4	5	0.015495	0.015751	0.017521

[141]:
```
plt.figure(figsize=(8, 5))
plt.plot(compare_curve["month_on_book"], compare_curve["observed_cum_pd"],␣
 ↪label="Observed")
plt.plot(compare_curve["month_on_book"], compare_curve["cum_pd_logit"],␣
 ↪label="Logistic hazard")
plt.plot(compare_curve["month_on_book"], compare_curve["cum_pd_rf"],␣
 ↪label="Random forest")
plt.xlabel("Month on book")
plt.ylabel("Cumulative default probability")
plt.title("Observed vs Predicted Cumulative Default")
plt.legend()
plt.show()
```



[142]:
```
milestones = [6, 12, 18, 24]

calibration_table = compare_curve[
    compare_curve["month_on_book"].isin(milestones)
].copy()

calibration_table["logit_error"] = (
    calibration_table["cum_pd_logit"] - calibration_table["observed_cum_pd"]
)

calibration_table["rf_error"] = (
    calibration_table["cum_pd_rf"] - calibration_table["observed_cum_pd"]
)

calibration_table
```

[142]:
```
    month_on_book  observed_cum_pd  cum_pd_logit  cum_pd_rf  logit_error  \
5               6         0.019200      0.019812   0.021988     0.000612
11             12         0.040133      0.055310   0.056448     0.015177
17             18         0.050575      0.095904   0.094783     0.045329
23             24         0.052404      0.133549   0.135185     0.081145

    rf_error
```

```
5   0.002788
11  0.016316
17  0.044208
23  0.082781
```

[143]:
```python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from lifelines import KaplanMeierFitter

# ==========================================================
# 1. Build loan-level TEST set for Kaplan-Meier
# ==========================================================

# Make sure issue_d is datetime
df["issue_d"] = pd.to_datetime(df["issue_d"], errors="coerce")

# Match your test split
test_loans = df[df["issue_d"] >= "2017-01-01"].copy()

# Keep one row per loan with survival inputs
loan_obs = (
    test_loans[["id", "duration_months", "event"]]
    .drop_duplicates(subset=["id"])
    .copy()
)

# Clean types
loan_obs["duration_months"] = pd.to_numeric(loan_obs["duration_months"],␣
 ↪errors="coerce")
loan_obs["event"] = pd.to_numeric(loan_obs["event"], errors="coerce")

# Drop bad rows if any
loan_obs = loan_obs.dropna(subset=["id", "duration_months", "event"]).copy()

# Optional: force valid ranges/types
loan_obs["duration_months"] = loan_obs["duration_months"].astype(int)
loan_obs["event"] = loan_obs["event"].astype(int)

print("Loan-level test sample shape:", loan_obs.shape)
print(loan_obs.head())


# ==========================================================
# 2. Fit Kaplan-Meier on observed test-loan outcomes
# ==========================================================
```

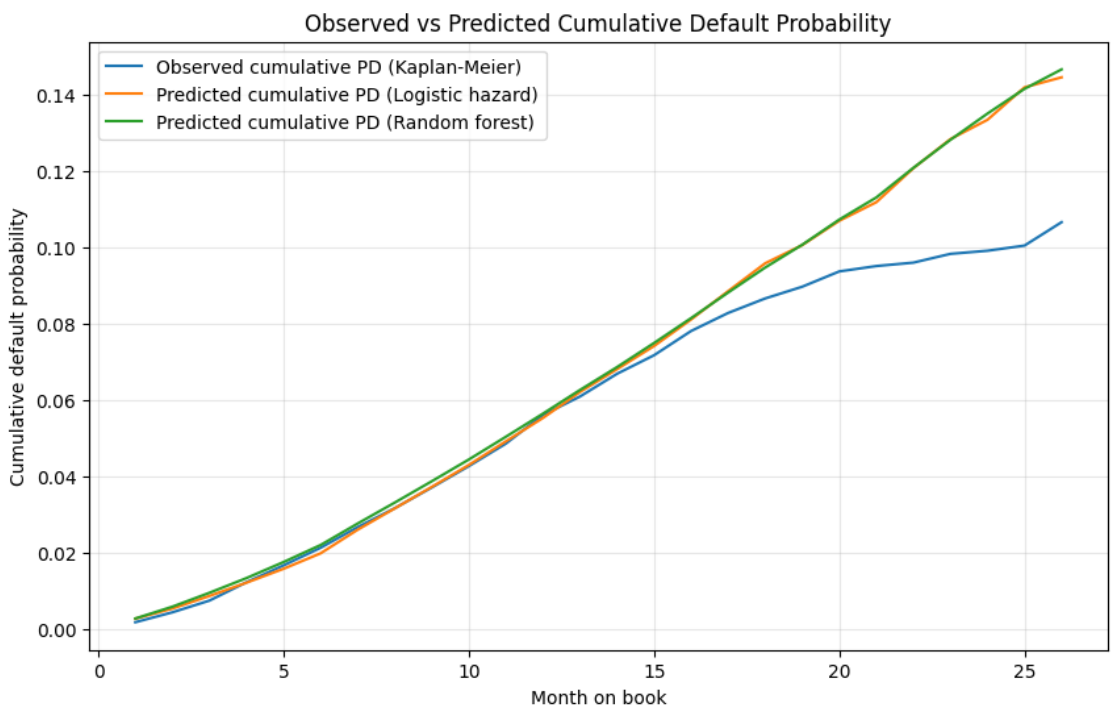
```python
kmf = KaplanMeierFitter()

kmf.fit(
    durations=loan_obs["duration_months"],
    event_observed=loan_obs["event"],
    label="Kaplan-Meier survival"
)

# Convert survival curve to cumulative default curve
km_curve = kmf.survival_function_.reset_index()
km_curve.columns = ["month_on_book", "survival_prob"]
km_curve["observed_cum_pd_km"] = 1 - km_curve["survival_prob"]

print("\nKaplan-Meier curve:")
print(km_curve.head(15))


# ============================================================
# 3. Build average model-predicted cumulative PD curve
# ============================================================

plot_cum_pd = (
    test_scored.groupby("month_on_book")[["cum_pd_logit", "cum_pd_rf"]]
    .mean()
    .reset_index()
    .sort_values("month_on_book")
)

print("\nPredicted cumulative PD curve:")
print(plot_cum_pd.head(15))


# ============================================================
# 4. Merge KM observed curve with model curves
# ============================================================

compare_curve = pd.merge(
    plot_cum_pd,
    km_curve[["month_on_book", "observed_cum_pd_km"]],
    on="month_on_book",
    how="left"
).sort_values("month_on_book")

print("\nMerged comparison curve:")
print(compare_curve.head(15))
```

```python
# ============================================================
# 5. Plot observed KM cumulative default vs predicted curves
# ============================================================

plt.figure(figsize=(10, 6))
plt.plot(
    compare_curve["month_on_book"],
    compare_curve["observed_cum_pd_km"],
    label="Observed cumulative PD (Kaplan-Meier)"
)
plt.plot(
    compare_curve["month_on_book"],
    compare_curve["cum_pd_logit"],
    label="Predicted cumulative PD (Logistic hazard)"
)
plt.plot(
    compare_curve["month_on_book"],
    compare_curve["cum_pd_rf"],
    label="Predicted cumulative PD (Random forest)"
)

plt.xlabel("Month on book")
plt.ylabel("Cumulative default probability")
plt.title("Observed vs Predicted Cumulative Default Probability")
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()


# ============================================================
# 6. Optional: milestone calibration table
# ============================================================

milestones = [6, 12, 18, 24]

calibration_table = (
    compare_curve[compare_curve["month_on_book"].isin(milestones)]
    .copy()
)

calibration_table["logit_error"] = (
    calibration_table["cum_pd_logit"] - calibration_table["observed_cum_pd_km"]
)

calibration_table["rf_error"] = (
    calibration_table["cum_pd_rf"] - calibration_table["observed_cum_pd_km"]
)
```

```python
print("\nCalibration table:")
print(calibration_table[[
    "month_on_book",
    "observed_cum_pd_km",
    "cum_pd_logit",
    "cum_pd_rf",
    "logit_error",
    "rf_error"
]])
```

```
Loan-level test sample shape: (20781, 3)
                id  duration_months  event
2066630  125356772               10      0
477199   128490686               13      0
641990   109655666               14      0
507598   127049010               14      0
2129008  121746633               17      0

Kaplan-Meier curve:
    month_on_book  survival_prob  observed_cum_pd_km
0             0.0       1.000000            0.000000
1             1.0       0.998268            0.001732
2             2.0       0.995668            0.004332
3             3.0       0.992590            0.007410
4             4.0       0.987773            0.012227
5             5.0       0.983358            0.016642
6             6.0       0.978742            0.021258
7             7.0       0.973322            0.026678
8             8.0       0.968358            0.031642
9             9.0       0.962924            0.037076
10           10.0       0.957323            0.042677
11           11.0       0.951409            0.048591
12           12.0       0.943901            0.056099
13           13.0       0.939060            0.060940
14           14.0       0.933096            0.066904

Predicted cumulative PD curve:
    month_on_book  cum_pd_logit  cum_pd_rf
0               1      0.002654   0.002710
1               2      0.005313   0.005819
2               3      0.008619   0.009456
3               4      0.012111   0.013319
4               5      0.015751   0.017521
5               6      0.019812   0.021988
6               7      0.025994   0.027624
7               8      0.031587   0.033112
8               9      0.037233   0.038714
```

```
9              10      0.042987   0.044419
10             11      0.049193   0.050388
11             12      0.055310   0.056448
12             13      0.062126   0.062659
13             14      0.068101   0.068664
14             15      0.074186   0.074989

Merged comparison curve:
    month_on_book  cum_pd_logit  cum_pd_rf  observed_cum_pd_km
0               1      0.002654   0.002710            0.001732
1               2      0.005313   0.005819            0.004332
2               3      0.008619   0.009456            0.007410
3               4      0.012111   0.013319            0.012227
4               5      0.015751   0.017521            0.016642
5               6      0.019812   0.021988            0.021258
6               7      0.025994   0.027624            0.026678
7               8      0.031587   0.033112            0.031642
8               9      0.037233   0.038714            0.037076
9              10      0.042987   0.044419            0.042677
10             11      0.049193   0.050388            0.048591
11             12      0.055310   0.056448            0.056099
12             13      0.062126   0.062659            0.060940
13             14      0.068101   0.068664            0.066904
14             15      0.074186   0.074989            0.071779
```



```
Calibration table:
    month_on_book  observed_cum_pd_km  cum_pd_logit  cum_pd_rf  logit_error  \
5               6            0.021258      0.019812   0.021988    -0.001447
11             12            0.056099      0.055310   0.056448    -0.000789
17             18            0.086677      0.095904   0.094783     0.009227
23             24            0.099196      0.133549   0.135185     0.034352

    rf_error
5   0.000730
11  0.000349
17  0.008106
23  0.035989
```

15 Variable importance

[144]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from copy import deepcopy

from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.calibration import CalibratedClassifierCV
from sklearn.metrics import (
    roc_auc_score,
    average_precision_score,
    brier_score_loss,
    log_loss,
)
from sklearn.inspection import permutation_importance
```

[145]:
```
target = "hazard_target"

all_numeric_features = [
    "loan_amnt",
    "term_num",
    "installment",
    "annual_inc",
    "dti",
    "delinq_2yrs",
    "inq_last_6mths",
```

```
    "open_acc",
    "pub_rec",
    "revol_bal",
    "revol_util",
    "total_acc",
    "mort_acc",
    "pub_rec_bankruptcies",
    "fico_avg",
    "credit_history_years",
    "fico_x_mob",
    "term_x_mob",
    "dti_x_mob",
]

all_categorical_features = [
    "emp_length",
    "home_ownership",
    "verification_status",
    "purpose",
    "addr_state",
    "mob_str",
    "mob_6bin",
]

# Group variables in a way that matches your reasoning
feature_groups = {
    "loan_size_terms": ["loan_amnt", "term_num", "installment"],
    "employment_home": ["emp_length", "home_ownership"],
    "income_verification": ["annual_inc", "verification_status"],
    "purpose_state": ["purpose", "addr_state"],
    "delinq_inquiry": ["delinq_2yrs", "inq_last_6mths"],
    "accounts_public": ["open_acc", "pub_rec", "pub_rec_bankruptcies"],
    "utilization_balance": ["revol_bal", "revol_util"],
    "capacity_depth": ["total_acc", "mort_acc", "credit_history_years"],
    "credit_quality": ["fico_avg", "dti"],
    "time_main": ["mob_str", "mob_6bin"],
    "time_interactions": ["fico_x_mob", "term_x_mob", "dti_x_mob"],
}

# Also test single raw features if you want finer simplification
single_feature_groups = {f: [f] for f in (all_numeric_features +␣
 ↪all_categorical_features)}
```

[146]:
```
def split_feature_types(features, numeric_pool, categorical_pool):
    num = [f for f in features if f in numeric_pool]
    cat = [f for f in features if f in categorical_pool]
    return num, cat
```

```python
def make_preprocessor(numeric_features, categorical_features):
    numeric_transformer = Pipeline([
        ("imputer", SimpleImputer(strategy="median")),
        ("scaler", StandardScaler()),
    ])

    categorical_transformer = Pipeline([
        ("imputer", SimpleImputer(strategy="most_frequent")),
        ("onehot", OneHotEncoder(handle_unknown="ignore")),
    ])

    preprocessor = ColumnTransformer([
        ("num", numeric_transformer, numeric_features),
        ("cat", categorical_transformer, categorical_features),
    ])
    return preprocessor


def make_logit_pipeline(numeric_features, categorical_features):
    preprocessor = make_preprocessor(numeric_features, categorical_features)
    model = Pipeline([
        ("prep", preprocessor),
        ("model", LogisticRegression(max_iter=2000))
    ])
    return model


def make_rf_pipeline(numeric_features, categorical_features, random_state=42):
    preprocessor = make_preprocessor(numeric_features, categorical_features)

    rf_base = Pipeline([
        ("prep", preprocessor),
        ("model", RandomForestClassifier(
            n_estimators=200,
            min_samples_leaf=50,
            random_state=random_state,
            n_jobs=-1,
            class_weight="balanced_subsample"
        ))
    ])

    rf_calibrated = CalibratedClassifierCV(
        estimator=rf_base,
        method="sigmoid",
        cv=3
```

```
    )
    return rf_calibrated


def get_pred_proba(model, X):
    return model.predict_proba(X)[:, 1]


def compute_metrics(y_true, y_prob, eps=1e-12):
    y_prob = np.clip(y_prob, eps, 1 - eps)
    return {
        "roc_auc": roc_auc_score(y_true, y_prob),
        "pr_auc": average_precision_score(y_true, y_prob),
        "brier": brier_score_loss(y_true, y_prob),
        "logloss": log_loss(y_true, y_prob),
        "mean_pred": float(np.mean(y_prob)),
        "event_rate": float(np.mean(y_true)),
    }


def fit_and_score_model(
    model_type,
    features,
    X_train_full, y_train,
    X_valid_full, y_valid,
    X_test_full, y_test,
    numeric_pool,
    categorical_pool,
    random_state=42
):
    numeric_features, categorical_features = split_feature_types(
        features, numeric_pool, categorical_pool
    )

    X_train = X_train_full[features].copy()
    X_valid = X_valid_full[features].copy()
    X_test = X_test_full[features].copy()

    if model_type == "logit":
        model = make_logit_pipeline(numeric_features, categorical_features)
    elif model_type == "rf":
        model = make_rf_pipeline(numeric_features, categorical_features,␣
 ↪random_state=random_state)
    else:
        raise ValueError("model_type must be 'logit' or 'rf'")

    model.fit(X_train, y_train)
```

```
    valid_prob = get_pred_proba(model, X_valid)
    test_prob = get_pred_proba(model, X_test)

    valid_metrics = compute_metrics(y_valid, valid_prob)
    test_metrics = compute_metrics(y_test, test_prob)

    out = {
        "model": model,
        "features": features,
        "n_features_raw": len(features),
        "numeric_features": numeric_features,
        "categorical_features": categorical_features,
        "valid_metrics": valid_metrics,
        "test_metrics": test_metrics,
        "valid_prob": valid_prob,
        "test_prob": test_prob,
    }
    return out
```

[147]:
```
all_features = all_numeric_features + all_categorical_features

X_train_full = train_panel[all_features].copy()
y_train = train_panel[target].copy()

X_valid_full = valid_panel[all_features].copy()
y_valid = valid_panel[target].copy()

X_test_full = test_panel[all_features].copy()
y_test = test_panel[target].copy()
```

[148]:
```
baseline_logit = fit_and_score_model(
    model_type="logit",
    features=all_features,
    X_train_full=X_train_full, y_train=y_train,
    X_valid_full=X_valid_full, y_valid=y_valid,
    X_test_full=X_test_full, y_test=y_test,
    numeric_pool=all_numeric_features,
    categorical_pool=all_categorical_features,
)

baseline_rf = fit_and_score_model(
    model_type="rf",
    features=all_features,
    X_train_full=X_train_full, y_train=y_train,
    X_valid_full=X_valid_full, y_valid=y_valid,
    X_test_full=X_test_full, y_test=y_test,
```

```
    numeric_pool=all_numeric_features,
    categorical_pool=all_categorical_features,
)

print("Baseline logit VALID:", baseline_logit["valid_metrics"])
print("Baseline logit TEST :", baseline_logit["test_metrics"])
print()
print("Baseline RF VALID   :", baseline_rf["valid_metrics"])
print("Baseline RF TEST    :", baseline_rf["test_metrics"])
```

```
Baseline logit VALID: {'roc_auc': 0.6707368973775432, 'pr_auc':
0.014084952061406895, 'brier': 0.00689730385618504, 'logloss':
0.04031133921256094, 'mean_pred': 0.006319581416688466, 'event_rate':
0.006962201922153251}
Baseline logit TEST : {'roc_auc': 0.6764559575734034, 'pr_auc':
0.008912978406677879, 'brier': 0.005109826723854781, 'logloss':
0.03395321397452347, 'mean_pred': 0.0058008378170974996, 'event_rate':
0.0045074236276504845}

Baseline RF VALID   : {'roc_auc': 0.6509492518507828, 'pr_auc':
0.01297307713652124, 'brier': 0.006899304983699516, 'logloss':
0.04055245662575222, 'mean_pred': 0.006298076060438385, 'event_rate':
0.006962201922153251}
Baseline RF TEST    : {'roc_auc': 0.6668219845528852, 'pr_auc':
0.008616006069964852, 'brier': 0.004480140348208971, 'logloss':
0.02814383980989728, 'mean_pred': 0.005135004991724497, 'event_rate':
0.0045074236276504845}
```

[149]:
```
def run_group_removal_experiments(
    model_type,
    feature_groups,
    base_features,
    X_train_full, y_train,
    X_valid_full, y_valid,
    X_test_full, y_test,
    numeric_pool,
    categorical_pool,
    random_state=42
):
    baseline = fit_and_score_model(
        model_type=model_type,
        features=base_features,
        X_train_full=X_train_full, y_train=y_train,
        X_valid_full=X_valid_full, y_valid=y_valid,
        X_test_full=X_test_full, y_test=y_test,
        numeric_pool=numeric_pool,
        categorical_pool=categorical_pool,
```

```
        random_state=random_state
    )

    rows = []

    for group_name, remove_vars in feature_groups.items():
        reduced_features = [f for f in base_features if f not in remove_vars]

        result = fit_and_score_model(
            model_type=model_type,
            features=reduced_features,
            X_train_full=X_train_full, y_train=y_train,
            X_valid_full=X_valid_full, y_valid=y_valid,
            X_test_full=X_test_full, y_test=y_test,
            numeric_pool=numeric_pool,
            categorical_pool=categorical_pool,
            random_state=random_state
        )

        row = {
            "model_type": model_type,
            "removed_group": group_name,
            "removed_vars": ", ".join(remove_vars),
            "n_removed_raw": len(remove_vars),
            "n_features_remaining_raw": len(reduced_features),

            "baseline_valid_roc_auc": baseline["valid_metrics"]["roc_auc"],
            "baseline_valid_pr_auc": baseline["valid_metrics"]["pr_auc"],
            "baseline_valid_brier": baseline["valid_metrics"]["brier"],
            "baseline_valid_logloss": baseline["valid_metrics"]["logloss"],

            "reduced_valid_roc_auc": result["valid_metrics"]["roc_auc"],
            "reduced_valid_pr_auc": result["valid_metrics"]["pr_auc"],
            "reduced_valid_brier": result["valid_metrics"]["brier"],
            "reduced_valid_logloss": result["valid_metrics"]["logloss"],

            "delta_valid_roc_auc": result["valid_metrics"]["roc_auc"] -␣
↪baseline["valid_metrics"]["roc_auc"],
            "delta_valid_pr_auc": result["valid_metrics"]["pr_auc"] -␣
↪baseline["valid_metrics"]["pr_auc"],
            "delta_valid_brier": result["valid_metrics"]["brier"] -␣
↪baseline["valid_metrics"]["brier"],
            "delta_valid_logloss": result["valid_metrics"]["logloss"] -␣
↪baseline["valid_metrics"]["logloss"],

            "baseline_test_roc_auc": baseline["test_metrics"]["roc_auc"],
            "baseline_test_pr_auc": baseline["test_metrics"]["pr_auc"],
```

```
            "baseline_test_brier": baseline["test_metrics"]["brier"],
            "baseline_test_logloss": baseline["test_metrics"]["logloss"],

            "reduced_test_roc_auc": result["test_metrics"]["roc_auc"],
            "reduced_test_pr_auc": result["test_metrics"]["pr_auc"],
            "reduced_test_brier": result["test_metrics"]["brier"],
            "reduced_test_logloss": result["test_metrics"]["logloss"],

            "delta_test_roc_auc": result["test_metrics"]["roc_auc"] -␣
↪baseline["test_metrics"]["roc_auc"],
            "delta_test_pr_auc": result["test_metrics"]["pr_auc"] -␣
↪baseline["test_metrics"]["pr_auc"],
            "delta_test_brier": result["test_metrics"]["brier"] -␣
↪baseline["test_metrics"]["brier"],
            "delta_test_logloss": result["test_metrics"]["logloss"] -␣
↪baseline["test_metrics"]["logloss"],
        }
        rows.append(row)

    results_df = pd.DataFrame(rows).sort_values(
        ["delta_valid_pr_auc", "delta_valid_roc_auc"],
        ascending=[False, False]
    ).reset_index(drop=True)

    return baseline, results_df
```

[150]:
```
baseline_logit_grouped, grouped_logit_results = run_group_removal_experiments(
    model_type="logit",
    feature_groups=feature_groups,
    base_features=all_features,
    X_train_full=X_train_full, y_train=y_train,
    X_valid_full=X_valid_full, y_valid=y_valid,
    X_test_full=X_test_full, y_test=y_test,
    numeric_pool=all_numeric_features,
    categorical_pool=all_categorical_features,
)

baseline_rf_grouped, grouped_rf_results = run_group_removal_experiments(
    model_type="rf",
    feature_groups=feature_groups,
    base_features=all_features,
    X_train_full=X_train_full, y_train=y_train,
    X_valid_full=X_valid_full, y_valid=y_valid,
    X_test_full=X_test_full, y_test=y_test,
    numeric_pool=all_numeric_features,
    categorical_pool=all_categorical_features,
)
```

```
grouped_logit_results
```

[150]:

```
   model_type        removed_group                                removed_vars  \
0       logit       credit_quality                                fico_avg, dti
1       logit      employment_home                  emp_length, home_ownership
2       logit    time_interactions          fico_x_mob, term_x_mob, dti_x_mob
3       logit       delinq_inquiry                delinq_2yrs, inq_last_6mths
4       logit       capacity_depth  total_acc, mort_acc, credit_history_years
5       logit  income_verification            annual_inc, verification_status
6       logit      accounts_public    open_acc, pub_rec, pub_rec_bankruptcies
7       logit            time_main                           mob_str, mob_6bin
8       logit  utilization_balance                       revol_bal, revol_util
9       logit        purpose_state                         purpose, addr_state
10      logit      loan_size_terms           loan_amnt, term_num, installment

    n_removed_raw  n_features_remaining_raw  baseline_valid_roc_auc  \
0               2                        24                0.670737
1               2                        24                0.670737
2               3                        23                0.670737
3               2                        24                0.670737
4               3                        23                0.670737
5               2                        24                0.670737
6               3                        23                0.670737
7               2                        24                0.670737
8               2                        24                0.670737
9               2                        24                0.670737
10              3                        23                0.670737

    baseline_valid_pr_auc  baseline_valid_brier  baseline_valid_logloss  \
0                0.014085              0.006897                0.040311
1                0.014085              0.006897                0.040311
2                0.014085              0.006897                0.040311
3                0.014085              0.006897                0.040311
4                0.014085              0.006897                0.040311
5                0.014085              0.006897                0.040311
6                0.014085              0.006897                0.040311
7                0.014085              0.006897                0.040311
8                0.014085              0.006897                0.040311
9                0.014085              0.006897                0.040311
10               0.014085              0.006897                0.040311

    reduced_valid_roc_auc  reduced_valid_pr_auc  reduced_valid_brier  \
0                0.670749              0.014659             0.006894
1                0.673001              0.014418             0.006896
2                0.672170              0.014329             0.006895
3                0.668363              0.014179             0.006898
```

4	0.670934	0.014098	0.006898
5	0.669160	0.013978	0.006901
6	0.670332	0.013954	0.006898
7	0.655044	0.013944	0.006900
8	0.668676	0.013750	0.006896
9	0.665181	0.013597	0.006898
10	0.652216	0.012215	0.006905

	reduced_valid_logloss	delta_valid_roc_auc	delta_valid_pr_auc	\
0	0.040301	0.000012	0.000574	
1	0.040260	0.002264	0.000333	
2	0.040268	0.001433	0.000244	
3	0.040306	-0.002374	0.000094	
4	0.040294	0.000197	0.000013	
5	0.040284	-0.001577	-0.000107	
6	0.040341	-0.000405	-0.000131	
7	0.040564	-0.015693	-0.000141	
8	0.040355	-0.002061	-0.000335	
9	0.040389	-0.005556	-0.000488	
10	0.040588	-0.018521	-0.001870	

	delta_valid_brier	delta_valid_logloss	baseline_test_roc_auc	\
0	-2.808002e-06	-0.000011	0.676456	
1	-1.098669e-06	-0.000052	0.676456	
2	-2.319220e-06	-0.000043	0.676456	
3	3.501383e-07	-0.000005	0.676456	
4	4.156490e-07	-0.000017	0.676456	
5	3.394713e-06	-0.000027	0.676456	
6	8.845899e-07	0.000030	0.676456	
7	3.130511e-06	0.000253	0.676456	
8	-1.041919e-06	0.000044	0.676456	
9	1.070678e-06	0.000078	0.676456	
10	7.213730e-06	0.000277	0.676456	

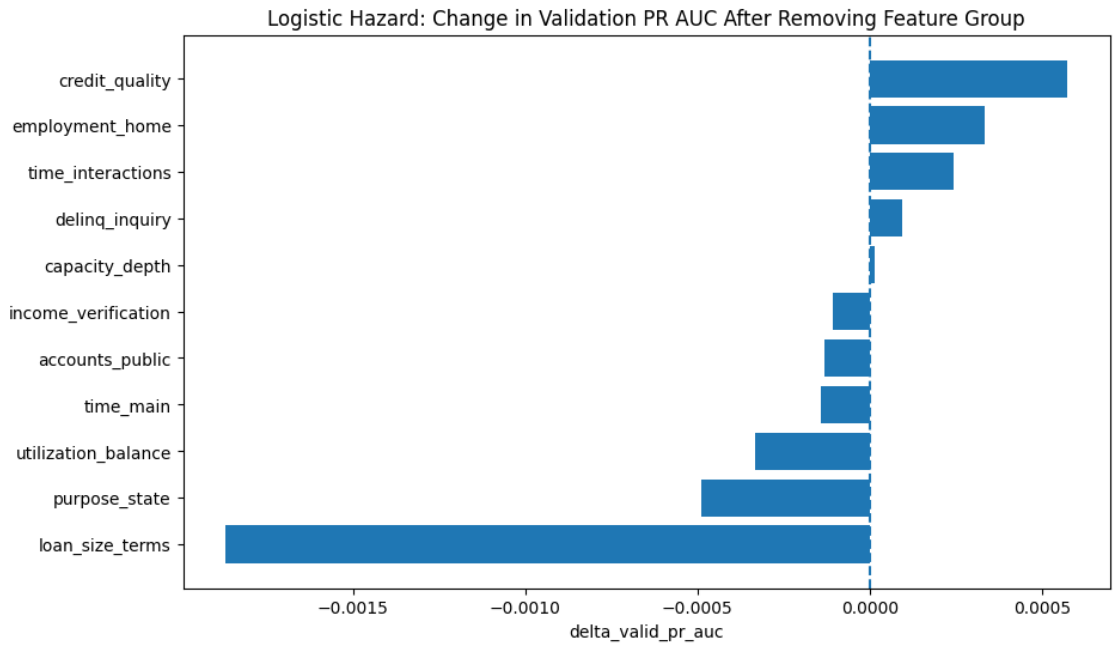
	baseline_test_pr_auc	baseline_test_brier	baseline_test_logloss	\
0	0.008913	0.00511	0.033953	
1	0.008913	0.00511	0.033953	
2	0.008913	0.00511	0.033953	
3	0.008913	0.00511	0.033953	
4	0.008913	0.00511	0.033953	
5	0.008913	0.00511	0.033953	
6	0.008913	0.00511	0.033953	
7	0.008913	0.00511	0.033953	
8	0.008913	0.00511	0.033953	
9	0.008913	0.00511	0.033953	
10	0.008913	0.00511	0.033953	

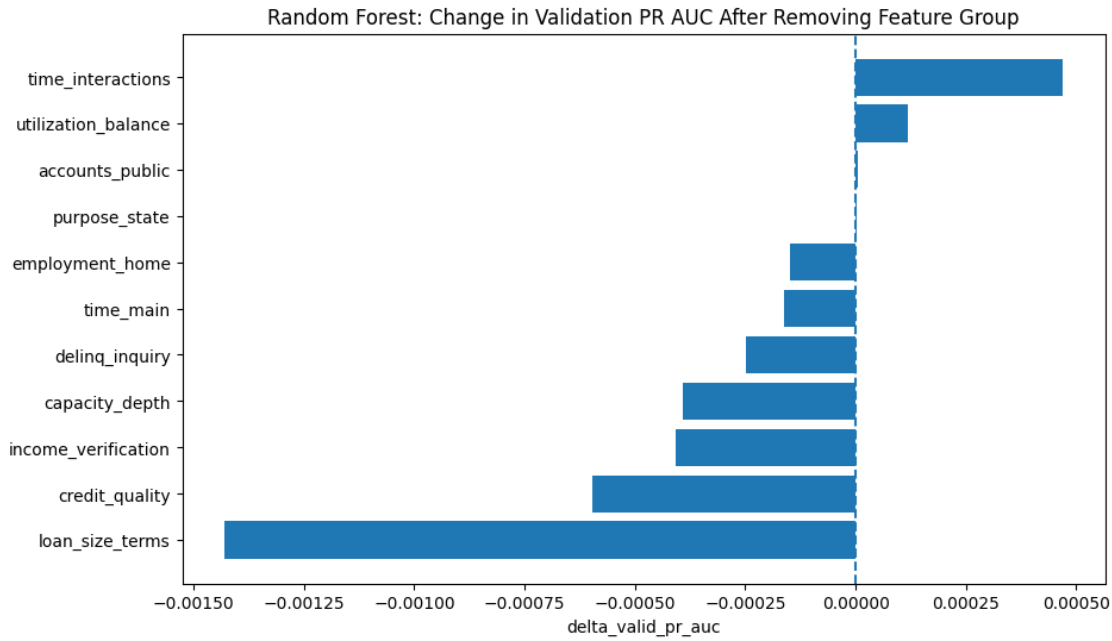
```
    reduced_test_roc_auc  reduced_test_pr_auc  reduced_test_brier  \
0               0.676218             0.009590            0.004579
1               0.678510             0.009151            0.005113
2               0.678170             0.009069            0.004990
3               0.671052             0.008754            0.005117
4               0.678380             0.008999            0.005117
5               0.676222             0.008936            0.005162
6               0.676116             0.008880            0.005116
7               0.668597             0.008616            0.005075
8               0.677646             0.009030            0.005000
9               0.669082             0.008818            0.005087
10              0.651675             0.007524            0.005221

    reduced_test_logloss  delta_test_roc_auc  delta_test_pr_auc  \
0               0.028454           -0.000238           0.000677
1               0.033944            0.002054           0.000238
2               0.032226            0.001714           0.000156
3               0.034089           -0.005404          -0.000159
4               0.033985            0.001924           0.000086
5               0.034940           -0.000234           0.000023
6               0.034059           -0.000340          -0.000033
7               0.033419           -0.007859          -0.000297
8               0.032310            0.001190           0.000117
9               0.033681           -0.007373          -0.000095
10              0.035830           -0.024781          -0.001389

    delta_test_brier  delta_test_logloss
0          -0.000531           -0.005499
1           0.000004           -0.000010
2          -0.000120           -0.001727
3           0.000008            0.000135
4           0.000008            0.000031
5           0.000052            0.000987
6           0.000006            0.000106
7          -0.000035           -0.000535
8          -0.000109           -0.001644
9          -0.000023           -0.000272
10          0.000111            0.001877
```

[151]:
```python
def plot_group_removal(results_df, metric_col, title):
    plot_df = results_df.sort_values(metric_col, ascending=True)

    plt.figure(figsize=(10, 6))
    plt.barh(plot_df["removed_group"], plot_df[metric_col])
    plt.axvline(0, linestyle="--")
    plt.xlabel(metric_col)
    plt.title(title)
```

```
    plt.show()
```

[152]:
```
plot_group_removal(
    grouped_logit_results,
    "delta_valid_pr_auc",
    "Logistic Hazard: Change in Validation PR AUC After Removing Feature Group"
)

plot_group_removal(
    grouped_rf_results,
    "delta_valid_pr_auc",
    "Random Forest: Change in Validation PR AUC After Removing Feature Group"
)
```





[ ]:
```
plot_group_removal(
    grouped_logit_results,
    "delta_valid_logloss",
    "Logistic Hazard: Change in Validation Log Loss After Removing Feature␣
 ↪Group"
)

plot_group_removal(
    grouped_rf_results,
    "delta_valid_logloss",
    "Random Forest: Change in Validation Log Loss After Removing Feature Group"
)
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[5], line 1
----> 1 plot_group_removal(
      2     grouped_logit_results,
      3     "delta_valid_logloss",
      4     "Logistic Hazard: Change in Validation Log Loss After Removing␣
 ↪Feature Group"
      5 )
      7 plot_group_removal(
      8     grouped_rf_results,
      9     "delta_valid_logloss",
```

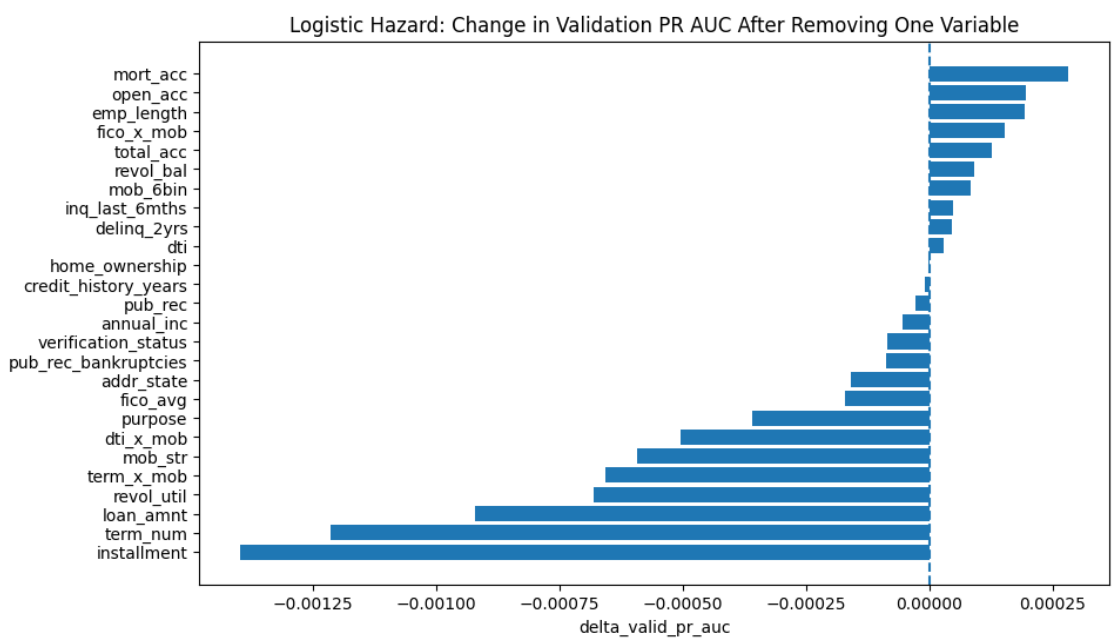
```
     10     "Random Forest: Change in Validation Log Loss After Removing Featur␣
 ↪Group"
     11 )

NameError: name 'plot_group_removal' is not defined
```

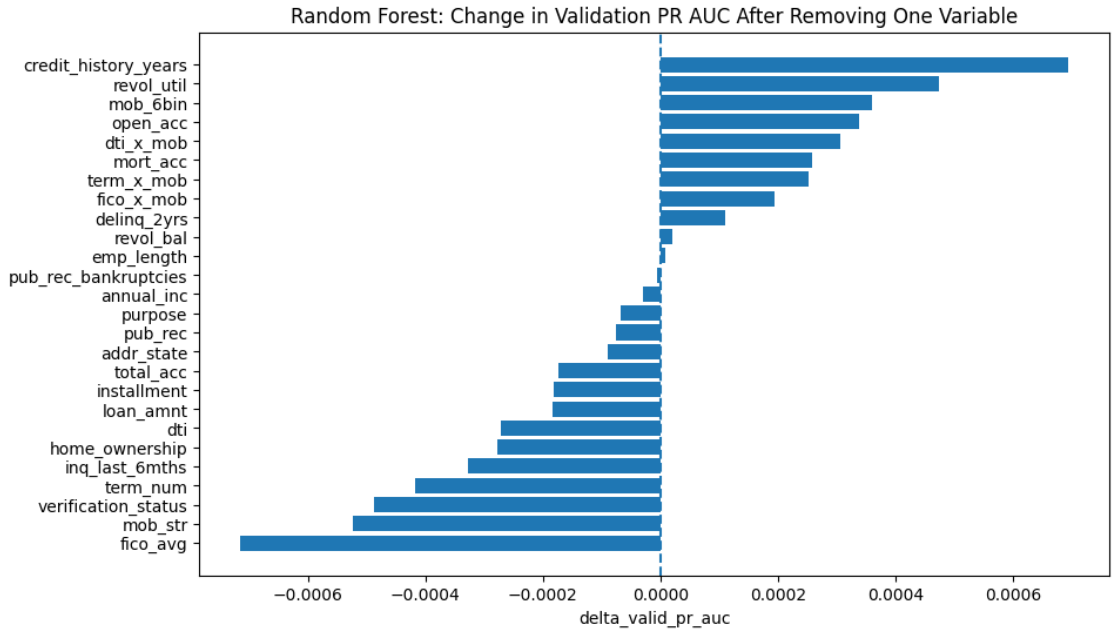
[4]:
```
baseline_logit_single, single_logit_results = run_group_removal_experiments(
    model_type="logit",
    feature_groups=single_feature_groups,
    base_features=all_features,
    X_train_full=X_train_full, y_train=y_train,
    X_valid_full=X_valid_full, y_valid=y_valid,
    X_test_full=X_test_full, y_test=y_test,
    numeric_pool=all_numeric_features,
    categorical_pool=all_categorical_features,
)

baseline_rf_single, single_rf_results = run_group_removal_experiments(
    model_type="rf",
    feature_groups=single_feature_groups,
    base_features=all_features,
    X_train_full=X_train_full, y_train=y_train,
    X_valid_full=X_valid_full, y_valid=y_valid,
    X_test_full=X_test_full, y_test=y_test,
    numeric_pool=all_numeric_features,
    categorical_pool=all_categorical_features,
)

single_logit_results.head(20)
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[4], line 1
----> 1 baseline_logit_single, single_logit_results =␣
 ↪run_group_removal_experiments(
      2     model_type="logit",
      3     feature_groups=single_feature_groups,
      4     base_features=all_features,
      5     X_train_full=X_train_full, y_train=y_train,
      6     X_valid_full=X_valid_full, y_valid=y_valid,
      7     X_test_full=X_test_full, y_test=y_test,
      8     numeric_pool=all_numeric_features,
      9     categorical_pool=all_categorical_features,
     10 )
     12 baseline_rf_single, single_rf_results = run_group_removal_experiments(
     13     model_type="rf",
```

```
     14     feature_groups=single_feature_groups,
   (…)      20     categorical_pool=all_categorical_features,
     21 )
     23 single_logit_results.head(20)

NameError: name 'run_group_removal_experiments' is not defined
```

[155]:
```
plot_group_removal(
    single_logit_results,
    "delta_valid_pr_auc",
    "Logistic Hazard: Change in Validation PR AUC After Removing One Variable"
)

plot_group_removal(
    single_rf_results,
    "delta_valid_pr_auc",
    "Random Forest: Change in Validation PR AUC After Removing One Variable"
)
```





[3]:
```
def flag_removal_candidates(results_df, pr_tol=0.002, logloss_tol=0.005):
    out = results_df.copy()

    out["safe_to_remove"] = (
        (out["delta_valid_pr_auc"] >= -pr_tol) &
        (out["delta_valid_logloss"] <= logloss_tol)
    )

    return out.sort_values(
        ["safe_to_remove", "delta_valid_pr_auc", "delta_valid_logloss"],
        ascending=[False, False, True]
    )

flagged_logit = flag_removal_candidates(grouped_logit_results)
flagged_rf = flag_removal_candidates(grouped_rf_results)

flagged_logit[[
    "removed_group", "removed_vars",
    "delta_valid_pr_auc", "delta_valid_roc_auc",
    "delta_valid_logloss", "safe_to_remove"
]]
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[3], line 14
      4     out["safe_to_remove"] = (
```

```
      5         (out["delta_valid_pr_auc"] >= -pr_tol) &
      6         (out["delta_valid_logloss"] <= logloss_tol)
      7     )
      9     return out.sort_values(
     10         ["safe_to_remove", "delta_valid_pr_auc", "delta_valid_logloss"]
     11         ascending=[False, False, True]
     12     )
---> 14 flagged_logit = flag_removal_candidates(grouped_logit_results)
     15 flagged_rf = flag_removal_candidates(grouped_rf_results)
     17 flagged_logit[[
     18     "removed_group", "removed_vars",
     19     "delta_valid_pr_auc", "delta_valid_roc_auc",
     20     "delta_valid_logloss", "safe_to_remove"
     21 ]]

NameError: name 'grouped_logit_results' is not defined
```

[157]:
```
groups_to_remove_logit = flagged_logit.loc[
    flagged_logit["safe_to_remove"], "removed_group"
].tolist()

groups_to_remove_logit
```

[157]:
```
['credit_quality',
 'employment_home',
 'time_interactions',
 'delinq_inquiry',
 'capacity_depth',
 'income_verification',
 'accounts_public',
 'time_main',
 'utilization_balance',
 'purpose_state',
 'loan_size_terms']
```

[161]:
```
groups_to_remove_logit = [
    "employment_home",
    "time_interactions",
    "capacity_depth",
]
remove_vars = []
for g in groups_to_remove_logit:
    remove_vars.extend(feature_groups[g])

remove_vars = sorted(set(remove_vars))

reduced_features_logit = [f for f in all_features if f not in remove_vars]
```

```
final_logit_reduced = fit_and_score_model(
    model_type="logit",
    features=reduced_features_logit,
    X_train_full=X_train_full, y_train=y_train,
    X_valid_full=X_valid_full, y_valid=y_valid,
    X_test_full=X_test_full, y_test=y_test,
    numeric_pool=all_numeric_features,
    categorical_pool=all_categorical_features,
)

print("BASELINE LOGIT VALID:", baseline_logit["valid_metrics"])
print("REDUCED  LOGIT VALID:", final_logit_reduced["valid_metrics"])
print()
print("BASELINE LOGIT TEST :", baseline_logit["test_metrics"])
print("REDUCED  LOGIT TEST :", final_logit_reduced["test_metrics"])
print()
print("Reduced features:", reduced_features_logit)
```

```
BASELINE LOGIT VALID: {'roc_auc': 0.6707368973775432, 'pr_auc':
0.014084952061406895, 'brier': 0.00689730385618504, 'logloss':
0.04031133921256094, 'mean_pred': 0.006319581416688466, 'event_rate':
0.006962201922153251}
REDUCED  LOGIT VALID: {'roc_auc': 0.6720058194754052, 'pr_auc':
0.014458438286886949, 'brier': 0.006897121324603765, 'logloss':
0.04030573120398532, 'mean_pred': 0.006253191705819508, 'event_rate':
0.006962201922153251}

BASELINE LOGIT TEST : {'roc_auc': 0.6764559575734034, 'pr_auc':
0.008912978406677879, 'brier': 0.005109826723854781, 'logloss':
0.03395321397452347, 'mean_pred': 0.0058008378170974996, 'event_rate':
0.0045074236276504845}
REDUCED  LOGIT TEST : {'roc_auc': 0.6813756710792058, 'pr_auc':
0.009296471022492578, 'brier': 0.005057158232846492, 'logloss':
0.033086543858152684, 'mean_pred': 0.005806059314088518, 'event_rate':
0.0045074236276504845}

Reduced features: ['loan_amnt', 'term_num', 'installment', 'annual_inc', 'dti',
'delinq_2yrs', 'inq_last_6mths', 'open_acc', 'pub_rec', 'revol_bal',
'revol_util', 'pub_rec_bankruptcies', 'fico_avg', 'verification_status',
'purpose', 'addr_state', 'mob_str', 'mob_6bin']
```

[165]:
```
# Observed monthly event rate in the test panel
observed_monthly = (
    test_panel.groupby("month_on_book")["hazard_target"]
    .mean()
    .reset_index()
```

```
    .rename(columns={"hazard_target": "observed_monthly_hazard"})
    .sort_values("month_on_book")
)

# Convert observed monthly hazard to observed cumulative PD
observed_monthly["observed_survival"] = (
    1 - observed_monthly["observed_monthly_hazard"]
).cumprod()

observed_monthly["observed_cum_pd"] = 1 - observed_monthly["observed_survival"]

observed_monthly.head()
```

[165]:

	month_on_book	observed_monthly_hazard	observed_survival	observed_cum_pd
0	1	0.001732	0.998268	0.001732
1	2	0.002604	0.995668	0.004332
2	3	0.003091	0.992590	0.007410
3	4	0.004853	0.987773	0.012227
4	5	0.004470	0.983358	0.016642

[1]:

```
def make_test_scored(panel_df, fitted_result, id_col="id",␣
 ↪month_col="month_on_book"):
    features = fitted_result["features"]
    model = fitted_result["model"]

    scored = panel_df.copy()
    scored["pred_hazard"] = get_pred_proba(model, panel_df[features])

    scored = scored.sort_values([id_col, month_col]).copy()
    scored["survival_prob"] = scored.groupby(id_col)["pred_hazard"].transform(
        lambda x: (1 - x).cumprod()
    )
    scored["cum_pd"] = 1 - scored["survival_prob"]
    return scored
```

[2]:

```
baseline_scored = make_test_scored(test_panel, baseline_logit)
reduced_scored = make_test_scored(test_panel, final_logit_reduced)

baseline_curve = baseline_scored.groupby("month_on_book")["cum_pd"].mean().
 ↪reset_index()
reduced_curve = reduced_scored.groupby("month_on_book")["cum_pd"].mean().
 ↪reset_index()

plt.figure(figsize=(9, 5))
plt.plot(compare_curve["month_on_book"], compare_curve["observed_cum_pd"],␣
 ↪label="Observed")
```

```
plt.plot(baseline_curve["month_on_book"], baseline_curve["cum_pd"],␣
 ↪label="Baseline logit")
plt.plot(reduced_curve["month_on_book"], reduced_curve["cum_pd"],␣
 ↪label="Reduced logit")
plt.xlabel("Month on book")
plt.ylabel("Average cumulative PD")
plt.title("Observed vs Baseline vs Reduced Logistic Hazard Model")
plt.legend()
plt.show()
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[2], line 1
----> 1 baseline_scored = make_test_scored(test_panel, baseline_logit)
      2 reduced_scored = make_test_scored(test_panel, final_logit_reduced)
      4 baseline_curve = baseline_scored.groupby("month_on_book")["cum_pd"].
 ↪mean().reset_index()

NameError: name 'test_panel' is not defined
```

[ ]:
```
logit_perm = permutation_importance(
    baseline_logit["model"],
    X_valid_full[baseline_logit["features"]],
    y_valid,
    scoring="average_precision",
    n_repeats=5,
    random_state=42,
    n_jobs=-1
)

perm_logit_df = pd.DataFrame({
    "feature": baseline_logit["features"],
    "importance_mean": logit_perm.importances_mean,
    "importance_std": logit_perm.importances_std,
}).sort_values("importance_mean", ascending=False)

perm_logit_df.head(20)
```

[164]:
```
plt.figure(figsize=(10, 6))
plot_df = perm_logit_df.sort_values("importance_mean", ascending=True).tail(20)
plt.barh(plot_df["feature"], plot_df["importance_mean"])
plt.xlabel("Permutation importance (Validation PR AUC decrease)")
plt.title("Logistic Hazard Permutation Importance")
plt.show()
```

```
---------------------------------------------------------------------------
```

```
NameError                                 Traceback (most recent call last)
Cell In[164], line 2
      1 plt.figure(figsize=(10, 6))
----> 2 plot_df = perm_logit_df.sort_values("importance_mean", ascending=True).
 ↪tail(20)
      3 plt.barh(plot_df["feature"], plot_df["importance_mean"])
      4 plt.xlabel("Permutation importance (Validation PR AUC decrease)")

NameError: name 'perm_logit_df' is not defined
```

```
<Figure size 1000x600 with 0 Axes>
```

[ ]:
```
rf_perm = permutation_importance(
    baseline_rf["model"],
    X_valid_full[baseline_rf["features"]],
    y_valid,
    scoring="average_precision",
    n_repeats=5,
    random_state=42,
    n_jobs=-1
)

perm_rf_df = pd.DataFrame({
    "feature": baseline_rf["features"],
    "importance_mean": rf_perm.importances_mean,
    "importance_std": rf_perm.importances_std,
}).sort_values("importance_mean", ascending=False)

perm_rf_df.head(20)
```

[ ]:
```
plt.figure(figsize=(10, 6))
plot_df = perm_rf_df.sort_values("importance_mean", ascending=True).tail(20)
plt.barh(plot_df["feature"], plot_df["importance_mean"])
plt.xlabel("Permutation importance (Validation PR AUC decrease)")
plt.title("Random Forest Permutation Importance")
plt.show()
```

[ ]:
```
from statsmodels.stats.outliers_influence import variance_inflation_factor
import statsmodels.api as sm
```

[ ]:
```
numeric_df = train_panel[all_numeric_features].copy()

# simple median fill for VIF calculation
numeric_df = numeric_df.fillna(numeric_df.median(numeric_only=True))

X_num = sm.add_constant(numeric_df)
```

```
vif_df = pd.DataFrame({
    "feature": X_num.columns,
    "VIF": [variance_inflation_factor(X_num.values, i) for i in range(X_num.
 ↪shape[1])]
}).sort_values("VIF", ascending=False)

vif_df
```

[ ]:
```
corr = numeric_df.corr()

plt.figure(figsize=(10, 8))
plt.imshow(corr, aspect="auto")
plt.colorbar()
plt.xticks(range(len(corr.columns)), corr.columns, rotation=90)
plt.yticks(range(len(corr.columns)), corr.columns)
plt.title("Correlation Matrix of Numeric Features")
plt.show()
```

[ ]:
```
def backward_elimination_by_pr_auc(
    model_type,
    start_features,
    X_train_full, y_train,
    X_valid_full, y_valid,
    X_test_full, y_test,
    numeric_pool,
    categorical_pool,
    tolerance=0.002,
    random_state=42
):
    current_features = start_features.copy()
    history = []

    while True:
        baseline = fit_and_score_model(
            model_type=model_type,
            features=current_features,
            X_train_full=X_train_full, y_train=y_train,
            X_valid_full=X_valid_full, y_valid=y_valid,
            X_test_full=X_test_full, y_test=y_test,
            numeric_pool=numeric_pool,
            categorical_pool=categorical_pool,
            random_state=random_state
        )

        candidates = []
        for f in current_features:
            reduced = [x for x in current_features if x != f]
```

```
            result = fit_and_score_model(
                model_type=model_type,
                features=reduced,
                X_train_full=X_train_full, y_train=y_train,
                X_valid_full=X_valid_full, y_valid=y_valid,
                X_test_full=X_test_full, y_test=y_test,
                numeric_pool=numeric_pool,
                categorical_pool=categorical_pool,
                random_state=random_state
            )

            candidates.append({
                "removed_feature": f,
                "delta_valid_pr_auc": result["valid_metrics"]["pr_auc"] -␣
↪baseline["valid_metrics"]["pr_auc"],
                "delta_valid_roc_auc": result["valid_metrics"]["roc_auc"] -␣
↪baseline["valid_metrics"]["roc_auc"],
                "delta_valid_logloss": result["valid_metrics"]["logloss"] -␣
↪baseline["valid_metrics"]["logloss"],
                "result": result,
            })

        cand_df = pd.DataFrame(candidates).sort_values(
            ["delta_valid_pr_auc", "delta_valid_roc_auc"],
            ascending=[False, False]
        ).reset_index(drop=True)

        best = cand_df.iloc[0]

        if best["delta_valid_pr_auc"] >= -tolerance:
            current_features.remove(best["removed_feature"])
            history.append({
                "removed_feature": best["removed_feature"],
                "delta_valid_pr_auc": best["delta_valid_pr_auc"],
                "delta_valid_roc_auc": best["delta_valid_roc_auc"],
                "delta_valid_logloss": best["delta_valid_logloss"],
                "n_features_remaining": len(current_features),
            })
        else:
            break

    final_result = fit_and_score_model(
        model_type=model_type,
        features=current_features,
        X_train_full=X_train_full, y_train=y_train,
        X_valid_full=X_valid_full, y_valid=y_valid,
```

```
        X_test_full=X_test_full, y_test=y_test,
        numeric_pool=numeric_pool,
        categorical_pool=categorical_pool,
        random_state=random_state
    )

    history_df = pd.DataFrame(history)
    return current_features, final_result, history_df
```

[ ]:
```
reduced_features_auto, reduced_result_auto, reduction_history =␣
 ↪backward_elimination_by_pr_auc(
    model_type="logit",
    start_features=all_features,
    X_train_full=X_train_full, y_train=y_train,
    X_valid_full=X_valid_full, y_valid=y_valid,
    X_test_full=X_test_full, y_test=y_test,
    numeric_pool=all_numeric_features,
    categorical_pool=all_categorical_features,
    tolerance=0.002
)

print("Final reduced features:", reduced_features_auto)
print()
print("Reduced VALID metrics:", reduced_result_auto["valid_metrics"])
print("Reduced TEST metrics :", reduced_result_auto["test_metrics"])
```

[ ]:
```
if len(reduction_history) > 0:
    plt.figure(figsize=(9, 5))
    plt.plot(range(1, len(reduction_history) + 1),␣
 ↪reduction_history["delta_valid_pr_auc"])
    plt.axhline(0, linestyle="--")
    plt.xlabel("Removal step")
    plt.ylabel("Delta validation PR AUC")
    plt.title("Backward Elimination Path")
    plt.show()

reduction_history
```